

Saturday, 20 December 2025

Swift basics

Enum

```
enum Direction {  
    case north, south, east, west  
}
```

Let move = Direction.east

Can hold values

```
Enum Result {  
    case success(data: String)  
    case failure(error: String)  
  
}
```

Protocol

```
Protocol Vehicle {  
    func start()  
}
```

```
Struct Car: Vehicle {  
    func start(){  
        print("Starting car")  
    }  
}
```

Extensions

```
extension Int{  
    func squared() -> Int {  
        return self*self  
    }  
}  
5.squared()
```

Error Handling

```
enum LoginError: Error{  
    case invalidUser  
}  
  
func login() throws {  
    throws LoginError.invalidUser  
}  
  
do {  
    try login()  
} catch{  
    print(error)  
}
```

Optionals

Basic

The variable may have a value or it may be nil

```
Var email: String? = null
```

```
Email = "manav@kapur.com"
```

Why? To prevent crashes like NullPointerException

Optional States

Has a value -> wrapped

Has no value -> nil

Unwrap Optionals

Forced Unwrapping (!)

```
Val name: String?= "manav"
```

```
print(name!)
```

If name == nil -> crash

Use only when you are 100% sure

Optional Binding (If let)

```
If let username = name {  
    print(username)  
} else {  
    print("no name")  
}
```

If optional has value -> assign to new variable

Else -> go to the else block

Guard let

```
Func login (user: String ){  
    guard let user = user else {  
        print("User missing")  
        return  
    }  
    print("Welcome \(user)")  
}
```

Why guard ? For handling nil immediately , after guard the variable is non optional

Nil Coalescing operator (??)

Short and sweet

```
Let username = user?? "Guest"
```

If name has value -> use it

If name is nil -> use guest

Optional chaining

Let light = person?.address?.street?.count

If any part is nil while expression = nil (no crash)

Implicitly Unwrapped Optionals (!?)

Var name : String!

It is an optional

But swift will auto unwrap it

If nil-> crash

Used in storyboards UIKit outlets like

@IBOutlets wear var label: UILabel!

Optional vs non options

A non optional must contain a value

A optional may contain nil, so code must safely unwrap it

Why optionals?

Swift optionals eliminate most null pointer crashes (NPEs)

By forcing the developer to unwrap values, Swift ensures type-safety and prevent runtime errors

How optionals work under the hood

```
Enum optional<wrappped> {
```

```
  Case none
```

```
  Case some(Wrapped)
```

```
}
```

```
So var name: String? = "Manav"
```

```
Optional.some("manav")
```

```
Var name: String? = nil
```

```
Optional.none
```

Struct vs class

Core

Struct. = Value type
Stored in stack
Copied when assigned or passed around

Class = Reference type
Stored in heap
Shared when assigned or passed around
Multiple variables can point to same instance

Example

```
Struct car {  
    var model: String  
}
```

```
Var c1 = car(model: "Thar")
```

```
Var c2 = c1;
```

```
c2.model = "Scorpio"
```

```
print(c1.model)  Thar
```

```
Print(c2.model)  Scorpio
```

C1 and c2 are copies

```
Class Bike{  
    Var mode: String  
    init(model: String){self.model = model}  
}
```

```
Var b1 = Bike(model: "Duke")
```

```
Var b2 = b1;
```

```
b2.model = "bullet"
```

```
print(b1.model) bullet
```

```
print(b2.model) bullet
```

B1 and b2 both reference the same object

Memory Concept

Struct are preferred because they are stored on stack making them lightweight, safer, eliminating side effects caused by shared mutable state

Stack

Fast

Lightweight

Copied

Class

Slow

Required ARC

Shared references

Mutability Difference

```
Struct Counter {  
    var count = 0  
    mutating func increment(){  
        count +=1  
    }  
}
```

```
Class Counter {  
    var count = 0  
    func increment(){  
        count+= 1  
    }  
}
```

Classes do not need mutating because classes reference shared data

Inheritance

Only classes support inheritance

```
Class Animal {}
```

```
Class Dog: Animal {}
```

Struct do not support inheritance

Both classes and structs can conform to protocols

ARC(Automatic reference counting)

Classes use ARC

Track numbers of references

Deallocate when count become zero

Can cause retain cycles

Use cases

Struct

Model objects, view models , pure data containers

Simple data

You want immutability

You want safer code without any side effects(retain cycle in ARC classes)

Value should be copied rather than shared

Example :- CGPoint, CGSize, CGRect

JSON model in SwiftUI and MVVM

Classes

You need inheritance

You need shared mutable data

You use objective-C APIS (UIKit heavily class based)

Object lifetime matters(deinit, ARC)

Example:- UIViewController, NSURLSession,UITableViewCell

How swiftUI uses Struct

SwiftUI redraws Ui often -> Struct make it fast

Immutable

Cheap to create

```
Struct ContentView: View{  
    var body: some View{  
        Text("hello")  
    }  
}
```

Swift UI handed performance :- Struct are copied by value, but swift use copy on write method. The actual memory is only copied when a mutation happen , making structures efficient

Protocols

Protocols are contracts that define behaviour without implementation. Swift encourages protocol oriented design , allowing structs, classes, enums to share behaviour. Protocol inheritance and associate types give swift huge flexibility without inheritance. Protocol make MVVM clean architecture, and swift UI testable and scalable.

Conforming to a protocol

A structure/class must implement all required methods

```
Protocol Vehicle{  
    func start()  
}  
  
Class car : Vehicle{  
    Func start(){  
        print("Car started")  
    }  
}
```

Protocols works with enum, struct and classes

```
Enum DeviceType: CustomStringConvertible {  
    case phone, tablet  
    var description: String { "\(self)" }  
}  
  
Let device: DeviceType = .phone  
print(Device) -> phone
```

Protocols property and method

```
Protocol Animal {
```

```
    var name: String { get}
```

```
    var age : String {get set}
```

```
    func makesound()
```

```
}
```

```
Struct Dog: Animal {
```

```
    let name: String = " Tommy"      //get
```

```
    var age: Int = 2                  //set requires var
```

```
    func makesound(){
```

```
        print("Bark")
```

```
    }
```

```
}
```

Protocol with initialisers

```
Protocol ViewModel {
```

```
    init()
```

```
}
```

```
Class HomeViewModel: ViewModel {
```

```
    required init() {}
```

```
}
```

Associated Types

Allow protocols to define placeholder that get specified by confirming type

```
Protocol Container {  
    associatedtype Item  
    func add(_ item: Item)  
}  
  
Struct IntStack: Container {  
    var items: [Int] = []  
    func add(_ item: Int){  
        print("Item \ \(item)")  
    }  
}
```

Protocol extension

```
Protocol Shape {  
    func area() -> Double  
}  
  
extension Shape{  
    func describe(){  
        print("I am in shape with area : \ \(area)")  
    }  
}
```

Any confirming type get function for free

Protocols in swift UI

```
Struct HomeView: View {  
    var body: some View{  
        Text("hello")  
    }  
}
```

```
Protocol View {  
    associated type Body: View  
    var body: Self.Body {get}  
}
```

Multiple Procol Conformance

```
struct Manager: Employable, Payable, Trackable {}
```

Use Protocol When

You want flexibility

No need for inheritance

You want dependency injection

You want testability

You want value types(structs) to conform

Use classes when

You need inheritance

You need shared mutable state

You need objective-C compatibility

Protocol Type vs Existential Any

```
struct EmailLoginVM: LoginViewModelProtocol {  
    var username: String  
    func login() { print("Email login") }  
}  
  
struct GoogleLoginVM: LoginViewModelProtocol {  
    var username: String  
    func login() { print("Google login") }  
}  
  
var vm: any LoginViewModelProtocol  
vm = EmailLoginVM(username: "manav")  
vm.login()  
vm = GoogleLoginVM(username: "manav")  
vm.login()
```

Q: Why is `any` slower?

A: Dynamic dispatch + type erasure + heap storage.

Q: When should you avoid `any`?

A: Performance-critical paths and generic algorithms.

Q: When should you use `any`?

A: Dependency injection, heterogeneous collections, ViewModel storage.

Closures

A closure is a function without a name , that you can store in a variable and passed around.

```
//Function as values
```

Basic Closure Example

```
//No parameter and no return value
```

```
Let greet = {  
    print("Hello Manav")  
}  
greet()
```

Closure with Parameters

```
//{(parameter)->returnType in  
    code  
}  
Let square = { (num: Int) -> Int in  
    return num * num  
}  
square(5)
```

Closure = Anonymous functions

```
func add(a: Int, b: Int) -> Int {  
    return a + b  
}
```

```
let add = {(a: Int, b: Int) -> Int in  
    a + b  
}
```

With Trailing closure Syntax

Trailing closure syntax allows you to write a closure outside the parenthesis of a function call when the closure is the last parameter

```
Func doSomething(completion: (Int) -> Void ){  
    DispatchQueue.global().async {  
        completion("Done")  
    }  
}  
  
fetchData { result in  
    print(result)  
}
```

Shorthand closure Syntax (\$0, \$1)

```
Let squared = { $0 * $0 }
```

```
squared(6)
```

```
Let numbers = [1,2,3]
```

```
Let double = numbers.map { $0 * $ 2 }
```

Closure Capture values

The closure remember variables outside its scope

This is called capture semantics

```
Var count = 0
```

```
Let increment = {
```

```
    count += 1
```

```
}
```

```
increment()
```

```
increment()
```

```
print(count) — > 2
```

Escaping closure

If a closure is used after the function returns , it must be marked as escaping.

```
Func fetchData(completion: @escaping (String) -> (void) ) {  
    DispatchQueue.global().async {  
        completion("Done")  
    }  
}
```

@escaping is required when the closure is execute after the function returns ,
Typically in async operation

Capture self in Closures(strong/weak)

We use [weak self] to avoid retain cycles when self is captured inside an escaping closure

```
Class ViewModel {  
    func load() {  
        API.getData { data in  
            self.handle(data)  
        }  
    }  
}
```

If self holds API and API holds closure -> retain cycle -> memory leak

```
API.getData { [weak self] data in  
    self?.handle(data)  
}
```

Closures in SwiftUI

```
Button("Tap"){  
    print("Tapped")  
}
```

UIKit – Complete Interview Guide

1. UIViewController Lifecycle (VERY IMPORTANT)

◆ Lifecycle Order

```
init
↓
loadView
↓
viewDidLoad
↓
viewWillAppear
↓
viewDidAppear
↓
(viewWillDisappear)
↓
(viewDidDisappear)
↓
deinit
```

◆ What to do WHERE?

Method	Purpose
viewDidLoad	One-time setup, API calls
viewWillAppear	Update UI every time
viewDidAppear	Animations, tracking
viewWillDisappear	Save state
deinit	Cleanup

✗ Common Mistake

Calling API in `viewWillAppear` → multiple calls

Interview Answer

viewDidLoad is called once, viewWillAppear is called every time the view appears.

2. UIKit UI Components

Common Components

- UILabel
- UIButton
- UITextField
- UITextView
- UIImageView
- UISwitch
- UISlider

Example

```
let label = UILabel()  
label.text = "Hello"  
label.textColor = .black
```

Interview Tip

UIKit follows **MVC** by default.

3. AutoLayout Constraints

What is AutoLayout?

Rule-based layout system that adapts to:

- Screen sizes

- Orientation
- Dynamic Type

◆ Constraint Types

- Width / Height
- Leading / Trailing
- Top / Bottom
- CenterX / CenterY

✗ Bad (Autoresizing)

```
view.frame = CGRect(x: 0, y: 0, width: 100, height: 100)
```

✓ AutoLayout

```
view.translatesAutoresizingMaskIntoConstraints = false  
view.centerXAnchor.constraint(equalTo:  
parent.centerXAnchor).isActive = true
```

🎯 Interview Answer

AutoLayout adapts UI across devices using constraints instead of frames.

4. Storyboards vs Programmatic UI

◆ Storyboards

- ✓ Fast prototyping
- ✗ Merge conflicts
- ✗ Hard to scale

◆ Programmatic UI

- ✓ Version control friendly
- ✓ Better scalability
- ✗ More code

Interview Answer

I prefer programmatic UI for scalability and clean architecture.

5. Delegates & DataSources

◆ What is Delegate?

One object **delegates responsibility** to another.

✓ Delegate Pattern

```
protocol LoginDelegate: AnyObject {  
    func didLogin()  
}
```

```
weak var delegate: LoginDelegate?
```

◆ DataSource

Provides **data** to UI components.

Interview Answer

Delegates handle behavior, data sources provide data.

6. UINavigationController & TabBarController

◆ UINavigationController

- Stack-based navigation

- Push / Pop

```
navigationController?.pushViewController(vc, animated: true)
```

◆ UITabBarController

- Parallel navigation
- Multiple root controllers

🎯 Interview Answer

NavigationController manages hierarchical flow, TabBarController manages parallel sections.

7. UITableView & UICollectionView

◆ UITableView

- Vertical list
- Reusable cells

◆ UICollectionView

- Grid / horizontal layouts
- Highly customizable

✅ TableView Example

```
func tableView(_ tableView: UITableView,
               cellForRowAt indexPath: IndexPath) ->
UITableViewCell {
    let cell =
tableView.dequeueReusableCell(withIdentifier: "Cell",
for: indexPath)
```

```
        return cell
    }
```

Interview Tip

Cell reuse is critical for performance.

8. Diffable Data Source (MODERN UIKit)

◆ Why Diffable Data Source?

- Automatic animations
- No crashes
- Cleaner updates

Example

```
var dataSource: UITableViewDiffableDataSource<Section,
Item>
```

```
var snapshot = NSDiffableDataSourceSnapshot<Section,
Item>()
snapshot.appendSections([.main])
snapshot.appendItems(items)
```

```
dataSource.apply(snapshot, animatingDifferences: true)
```

Interview Answer

Diffable Data Source simplifies data updates safely.

9. Gesture Recognizers

◆ Types

- Tap

- Pan
- Swipe
- Long press
- Pinch

Example

```
let tap = UITapGestureRecognizer(target: self, action:
#selector(handleTap))
view.addGestureRecognizer(tap)
```

Interview Answer

Gesture recognizers decouple input handling from view logic.

10. Layout Anchors (Preferred Way)

Why Layout Anchors?

- Safer
- Readable
- Type-safe

Example

```
button.leadingAnchor.constraint(equalTo:
view.leadingAnchor, constant: 16).isActive = true
```

Interview Tip

Layout anchors are preferred over NSLayoutConstraint initializers.

11. UIResponder Chain (ADVANCED, INTERVIEW GOLD)

◆ What is Responder Chain?

Chain of objects that can respond to events:

View → ViewController → Window → Application

◆ Handles

- Touches
- Motion
- Remote control events

🎯 Interview Answer

The responder chain determines which object handles user events.

12. Common UIKit Interview Traps

- ✗ Heavy work in `cellForRowAt`
- ✗ UI updates on background thread
- ✗ Missing `weak` delegates
- ✗ Forgetting reuse identifiers
- ✗ Constraint conflicts

🔥 Power Interview Answer

I'm comfortable with UIKit lifecycle, AutoLayout, delegates, navigation controllers, table and collection views, diffable data sources, gesture recognizers, and responder chain handling.

SwiftUI

SwiftUI – Complete Interview Guide

1. SwiftUI View Lifecycle

◆ Key Concept

SwiftUI is **state-driven**, not lifecycle-driven like UIKit.

There is **NO** `viewDidLoad`, `viewWillAppear`.

◆ Lifecycle Hooks

Modifier	When it runs
<code>init()</code>	View struct creation
<code>.onAppear</code>	View appears
<code>.onDisappear</code>	View disappears
<code>.task</code>	Async task on appear

✓ Example

```
struct HomeView: View {
    var body: some View {
        Text("Hello")
            .onAppear {
                print("Appeared")
            }
            .task {
                await loadData()
            }
    }
}
```

Interview Answer

SwiftUI lifecycle is driven by state changes, not view controller callbacks.

2. State Management (MOST IMPORTANT)

◆ @State

- Local, simple state
- Value types only

```
@State private var count = 0
```

✓ Use for UI-only state

✗ Don't share across views

◆ @StateObject

- Owns a reference-type object
- Created **once**

```
@StateObject var viewModel = HomeViewModel()
```

✓ Use in **root view**

◆ @ObservedObject

- Observes external object
- Does NOT own lifecycle

```
@ObservedObject var viewModel: HomeViewModel
```

◆ @Binding

- Two-way connection
- Child modifies parent state

```
@Binding var isOn: Bool
```

◆ @EnvironmentObject

- Global shared state

```
@EnvironmentObject var session: SessionManager
```

✗ Crashes if not injected

🧠 Interview Cheat Sheet

Wrapper	Ownership
@State	Local
@StateObject	Owns
@ObservedObject	Borrows
@Binding	Two-way
@EnvironmentObject	Global

🎯 Interview Answer

Use @StateObject in root views, @ObservedObject in child views.

3. NavigationStack & NavigationLink

◆ NavigationStack (iOS 16+)

- Replaces NavigationView
- Type-safe navigation

```
NavigationStack {
```



```
        NavigationLink("Go", value: 1)
    }
```

◆ Programmatic Navigation

```
@State private var path = NavigationPath()

NavigationStack(path: $path) {
    Button("Go") {
        path.append(1)
    }
}
```

🎯 Interview Answer

NavigationStack provides type-safe and predictable navigation.

4. MVVM in SwiftUI (CRITICAL)

◆ Structure

View → ViewModel → Repository → API

✅ ViewModel

```
@MainActor
class HomeViewModel: ObservableObject {
    @Published var users: [User] = []

    func load() async {
        users = await fetchUsers()
    }
}
```

✅ View

```

struct HomeView: View {
    @StateObject var vm = HomeViewModel()

    var body: some View {
        List(vm.users) { user in
            Text(user.name)
        }
        .task {
            await vm.load()
        }
    }
}

```

Interview Answer

SwiftUI MVVM is natural due to observable state binding.

5. Animations & Transitions

◆ Animations (State-based)

```

withAnimation {
    isExpanded.toggle()
}

```

◆ Transitions (View insertion/removal)

```

.transition(.opacity)

```

◆ Implicit Animation

```

.animation(.easeInOut, value: isExpanded)

```

Interview Answer

SwiftUI animations are declarative and driven by state changes.

6. Lists, LazyVGrid, LazyHGrid

◆ List

```
List(items) { item in  
    Text(item.title)  
}
```

◆ LazyVGrid

```
LazyVGrid(columns: [GridItem(.flexible())]) {  
    ForEach(items) { item in  
        Text(item.title)  
    }  
}
```

◆ Why “Lazy”?

- Views created **on demand**
- Memory efficient

🎯 Interview Answer

Lazy containers improve performance by creating views only when needed.

7. Custom View Modifiers

◆ Why?

- Reusable styling
- Clean UI code

✓ Custom Modifier

```
struct CardStyle: ViewModifier {  
    func body(content: Content) -> some View {  
        content  
        .padding()  
        .background(Color.white)  
        .cornerRadius(8)  
    }  
}
```

✓ Usage

```
Text("Hello")  
    .modifier(CardStyle())
```

🎯 Interview Answer

Custom modifiers promote reusable UI logic.

8. SwiftUI Layout System (INTERVIEW GOLD)

◆ Layout Order

1. Parent proposes size
2. Child chooses size
3. Parent positions child

◆ Common Containers

Container	Behavior
VStack	Vertical
HStack	Horizontal
ZStack	Overlapping

Spacer	Flexible space
--------	----------------

◆ Alignment & Priority

`.layoutPriority(1)`

🎯 Interview Answer

SwiftUI layout is proposal-based, not constraint-based.

9. Networking in SwiftUI

◆ Where networking goes?

✗ Not in View

✓ In ViewModel

✓ **async/await**

```
func loadUsers() async throws -> [User] {
    let (data, _) = try await
    URLSession.shared.data(from: url)
    return try JSONDecoder().decode([User].self, from:
data)
}
```

◆ Trigger from View

```
.task {
    await vm.load()
}
```

🎯 Interview Answer

SwiftUI networking is handled via async ViewModels.

Combine Basics (If Asked)

ObservableObject

```
class VM: ObservableObject {  
    @Published var text = ""  
}
```

Sink

```
publisher  
    .sink { value in }
```

When Combine is used?

- Form validation
- Search debounce
- Streams

Interview Answer

Combine is useful for reactive streams; async/await is better for single requests.

11. SwiftUI Common Interview Traps

-  Creating ViewModel with @ObservedObject in root
-  Heavy logic inside View
-  Missing @MainActor
-  Navigation state bugs
-  Overusing EnvironmentObject

Power Interview Answer

I use SwiftUI with MVVM, manage state using @StateObject and @ObservedObject, handle navigation via NavigationStack, perform networking using async/await in ViewModels, and optimize layouts with lazy containers and custom modifiers.

iOS Architecture – Complete Interview Guide

1. MVC (Model–View–Controller)

◆ What is MVC?

Apple's **default UIKit architecture**.

View ↔ Controller ↔ Model

◆ Roles

- **Model** → Business logic, data
- **View** → UI (UILabel, UIButton)
- **Controller** → Glue (UIViewController)

✗ Problem: Massive View Controller

```
class HomeVC: UIViewController {
    // UI
    // API calls
    // Parsing
    // Business logic
}
```

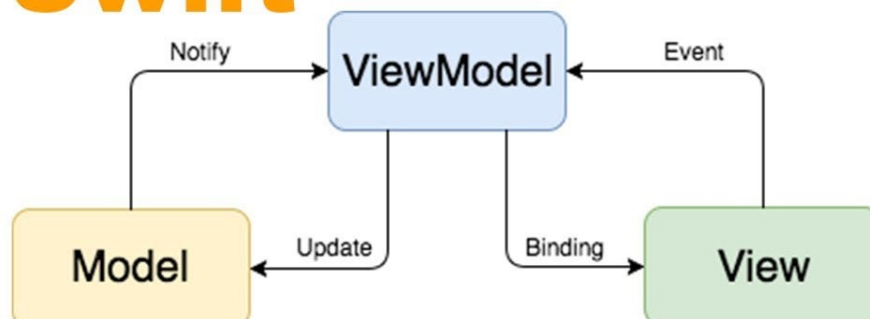
✓ Pros / ✗ Cons

Pros	Cons
Simple	Massive VC
Apple default	Hard to test
Quick	Tight coupling

🎯 Interview Answer

MVC is simple but doesn't scale well due to massive view controllers.

Model View ViewModel (MVVM) in Swift



2. MVVM (Most Used in Interviews)

MVVM = Model – View – ViewModel

It separates:

- **UI (View)**
- **Business logic (ViewModel)**
- **Data (Model)**

Why?

- Better testability
- Cleaner code
- Easy UI updates
- Avoids Massive View Controller (UIKit problem)

★ 2. MVVM Flow (Easy Explanation)

1 View

- SwiftUI screen
- Displays UI
- Observes ViewModel changes

2 ViewModel

- Contains logic
- Handles API calls
- Holds @Published vars
- Notifies the View when data changes

3 Model

- Data structure (struct)
- API response or domain model

★ 3. The SwiftUI MVVM Flow (VERY IMPORTANT)

User taps UI →
ViewModel updates state →
@Published notifies View →
SwiftUI refreshes UI →
New View reflects state →
This automatic UI-refresh is why SwiftUI works beautifully with MVVM.

★ 4. A Complete MVVM Example (Interview-Level)

🧱 Model

```
struct Quote: Decodable {  
    let text: String  
}
```

🧠 ViewModel

```
import Foundation  
  
class QuoteViewModel: ObservableObject {  
    @Published var quote: String = "Loading..."  
  
    func loadQuote() async {  
        // pretend API call  
        try? await Task.sleep(nanoseconds: 1_000_000_000)  
        quote = "Stay hungry, stay foolish."  
    }  
}
```

}

Key Concepts:

Keyword	Meaning
ObservableObject	Makes ViewModel observable
@Published	Updates the UI on change
async	Async call (modern Swift concurrency)

View

```
struct QuoteView: View {
    @StateObject private var vm = QuoteViewModel()

    var body: some View {
        VStack(spacing: 20) {
            Text(vm.quote)
                .font(.title)

            Button("Load Quote") {
                Task {
                    await vm.loadQuote()
                }
            }
        }
        .padding()
    }
}
```

Why @StateObject?

- Creates and **owns** the ViewModel
- Keeps ViewModel alive even when UI re-renders

★ 5. MVVM in UIKit (Samsung expects this too)

UIKit uses closures/delegates instead of @Published.

ViewController:

- Holds ViewModel
- Observes ViewModel updates manually

Example:

```
class LoginViewModel {  
    var username: String = "" {  
        didSet { onChange?(username) }  
    }  
  
    var onChange: ((String) -> Void)?  
}
```

ViewController:

```
vm.onChange = { [weak self] value in  
    self?.label.text = value  
}
```

★ 6. WHAT PROBLEM DOES MVVM SOLVE?

Interviewers ask this **all the time**.

Answer:

“MVVM removes logic from the View, keeps code testable, avoids Massive View Controller, and enables reactive UI updates.”

★ 7. Data Flow Variants (UIKit vs SwiftUI)

Framework	MVVM Style
UIKit	MVVM + Combine / Closures
SwiftUI	Pure MVVM using @StateObject, @ObservedObject

★ 8. Common MVVM Interview Questions (with answers)

? Q1: Why do we use MVVM?

It separates UI and logic, improves testability, readability, and reduces view controller bloat.

? Q2: What is the responsibility of a ViewModel?

Handle all business logic, API calls, state updates, input validation, and provide data to the View.

? Q3: Does ViewModel know about the View?

No.

View observes the ViewModel.

ViewModel should NEVER import SwiftUI or UIKit.

? Q4: What is @Published?

A property wrapper that automatically notifies the View when its value changes.

? Q5: When do you use @StateObject vs @ObservedObject?

Attribute	Usage
@StateObject	For creating a new ViewModel instance
@ObservedObject	When ViewModel is passed from parent

? Q6: MVVM vs MVC?

MVC	MVVM
ViewController does everything	ViewModel separates logic from View
Tight coupling	Loose coupling

Hard to test	Very testable
UIKit-style	Modern Apple recommended

★ 9. MVVM + Clean Architecture (Samsung Will Love This)

In enterprise apps, MVVM is used inside Clean Architecture:

Presentation Layer → SwiftUI View + ViewModel
 Domain Layer → Use Cases
 Data Layer → Repository, API, DB
 ViewModel does **not** call API directly.

Instead:

ViewModel → UseCase → Repository → API/DB
 Cleaner. Testable. Scalable.

★ 10. MVVM Best Practices (mention these in interview)

- ViewModel uses **@Published** for state
- View does NOT contain business logic
- ViewModel should not know View
- Avoid massive ViewModels
- Use protocol abstraction for testability
- Inject dependencies (APIService, Repository, UseCase)
- Keep Model pure
- Use main thread for UI updates



Summary You Can Tell in Interview

“MVVM splits the app into Model, View, and ViewModel.
The ViewModel exposes reactive state using @Published or Combine.
The View observes it and automatically updates.
This makes the code modular, testable, and scalable — perfect for enterprise-level iOS apps.”

◆ Binding

- UIKit → Closures / Combine
- SwiftUI → @StateObject, @ObservedObject

✓ Pros / ✗ Cons

Pros	Cons
Testable	Boilerplate
Clean separation	Binding complexity
Scalable	

🎯 Interview Answer

MVVM improves testability and separation of concerns.

3. VIPER (Enterprise-Level)

◆ What is VIPER?

VIPER is an architecture pattern derived from Clean Architecture. It splits responsibilities into **5 components**:

V I P E R

- **V** → View
- **I** → Interactor
- **P** → Presenter
- **E** → Entity
- **R** → Router

It is:

- Highly modular
- Extremely scalable
- Great for large enterprise apps

- Difficult for beginners (but you'll understand it easily now)

★ 2. Why VIPER Exists?

Because MVC and MVVM can grow massive:

- Massive ViewControllers
- Business logic leaking into UI
- Hard to test
- Hard to scale

VIPER solves this by splitting responsibilities clearly.

★ 3. VIPER Module Breakdown (The Heart of VIPER)

Here's what each component does:

✓ 1. View

- Shows UI
- Sends user events to Presenter
- Doesn't contain logic

✓ 2. Interactor

- Contains **business logic**
- Calls UseCases or repositories
- Similar to Clean Architecture's UseCase

✓ 3. Presenter

- Coordinates everything
- Builds the view state
- Receives user actions from View

- Tells Interactor what to do
- Updates View with results

✓ 4. Entity

- Basic data models
- Pure Swift structs/classes (no logic)

✓ 5. Router (a.k.a Wireframe)

- Handles navigation between screens
- Decides which screen to show next

★ 4. VIPER Flow (EXACT interview explanation)

```
View → Presenter → Interactor → Presenter → View
                ↓
                Entity
```

Navigation:

Presenter → Router → Next Screen

Interview-answer summary:

“The View sends events to the Presenter.
 The Presenter sends requests to the Interactor.
 The Interactor returns data to the Presenter.
 The Presenter updates the View.
 Router handles navigation.”

★ 5. A Simple VIPER Module Example

(VERY helpful for interviews)

● Entity

```
struct User {
    let id: Int
    let name: String
}
```

Interactor

```
protocol UserListInteractorProtocol {
    func fetchUsers()
}

final class UserListInteractor:
    UserListInteractorProtocol {
    var presenter: UserListPresenterProtocol?

    func fetchUsers() {
        // pretend API call
        let users = [
            User(id: 1, name: "Manav"),
            User(id: 2, name: "Aman")
        ]
        presenter?.didFetchUsers(users)
    }
}
```

Presenter

```
protocol UserListPresenterProtocol {
    func viewDidLoad()
    func didFetchUsers(_ users: [User])
}

final class UserListPresenter: UserListPresenterProtocol
{
    weak var view: UserListViewProtocol?
    var interactor: UserListInteractorProtocol?
    var router: UserListRouterProtocol?

    func viewDidLoad() {
```

```

        interactor?.fetchUsers()
    }

    func didFetchUsers(_ users: [User]) {
        let names = users.map { $0.name }
        view?.showUsers(names)
    }
}

```

View (UIKit example)

```

protocol UserListViewProtocol: AnyObject {
    func showUsers(_ users: [String])
}

final class UserListViewController: UIViewController,
UserListViewProtocol {
    var presenter: UserListPresenterProtocol?

    override func viewDidLoad() {
        super.viewDidLoad()
        presenter?.viewDidLoad()
    }

    func showUsers(_ users: [String]) {
        print("Users:", users)
    }
}

```

Router (Navigation)

```

protocol UserListRouterProtocol {
    static func createModule() -> UIViewController
}

final class UserListRouter: UserListRouterProtocol {
    static func createModule() -> UIViewController {
        let view = UserListViewController()
        let presenter = UserListPresenter()
        let interactor = UserListInteractor()
    }
}

```

```

        let router = UserListRouter()

        // wiring
        view.presenter = presenter
        presenter.view = view
        presenter.interactor = interactor
        presenter.router = router
        interactor.presenter = presenter

        return view
    }
}

```

★ 6. Why is VIPER so powerful?

✓ 1. Perfect separation of responsibility

Everything has a clear job.

✓ 2. Highly testable

Presenter, Interactor, Router can be tested individually.

✓ 3. Modular

Each screen = one VIPER module.

✓ 4. No Massive ViewController

All logic moves to Presenter and Interactor.

★ 7. VIPER vs MVVM (Interview Cheat Sheet)

Feature	MVVM	VIPER
Layers	View, ViewModel, Model	View, Interactor, Presenter, Entity, Router
Complexity	Medium	High
Best For	Simple/medium apps	Large enterprise modules
Navigation	Usually in View	Router handles navigation

Testability	High	Very high
SwiftUI-friendly	Yes	Not natural
UIKit enterprise apps	Good	Best

Perfect answer:

“VIPER is like MVVM but more strictly separated and more suitable for large apps that need modularity and strong test boundaries.”

8. Common VIPER Interview Questions (with perfect answers)

Q1: Why use VIPER?

VIPER enforces strict boundaries and separation of concerns, making large apps more maintainable and testable.

Q2: What are the roles of Presenter and Interactor?

Presenter:

Handles view logic, prepares data, mediates between View and Interactor.

Interactor:

Contains business logic and talks to data sources.

Q3: Who handles navigation?

Router (or Wireframe).

This separates navigation logic from business logic.

Q4: Can View talk directly to Interactor?

No.

View → Presenter → Interactor only.

Q5: Is VIPER suitable for SwiftUI?

Answer honestly:

VIPER fits UIKit better.

SwiftUI + MVVM + Clean Architecture is the modern approach.

9. When Should You Use VIPER?

Use VIPER when:

- App is large (50+ screens)
- Many teams working on different modules
- High testability required
- Screens must be independent
- Navigation is complex
- Strong separation is needed

Samsung's enterprise apps often fall in this category.

10. One-line VIPER Summary (Use in interview)

“VIPER splits UI, presentation, business logic, models, and navigation into five components. It improves scalability, testability, and eliminates Massive View Controllers, making it ideal for enterprise apps.”

5. Dependency Injection (CRITICAL)

Dependency Injection (DI)

◆ What is Dependency Injection?

Dependency Injection means **providing dependencies from outside** a class instead of **creating them internally**.

📌 Goal: **Loose coupling, testability, flexibility**

✗ Without DI (Tightly Coupled)

```
class HomeVM {  
    let api = APIService() // ✗ hard dependency  
}
```

✗ Problems:

- Cannot mock APIService
- Hard to test
- Class controls dependency creation

✓ With DI (Best Practice)

```
protocol APIServiceProtocol {  
    func fetchData()  
}
```



```

}

class HomeVM {
    let api: APIServiceProtocol

    init(api: APIServiceProtocol) {
        self.api = api
    }
}

```

- ✓ Easily mockable
- ✓ Replace implementation anytime
- ✓ Cleaner architecture

Testing Benefit (Interview Favorite)

```

class MockAPIService: APIServiceProtocol {
    func fetchData() {
        print("Mock data")
    }
}

```

```
let vm = HomeVM(api: MockAPIService())
```

Types of Dependency Injection (Know This)

Type	How
Constructor Injection	via <code>init</code>  (best)
Property Injection	set after init
Method Injection	passed per function

DI Interview One-Liner (Perfect)

“Dependency Injection improves testability and flexibility by decoupling object creation from usage.”

Coordinator Pattern (Navigation Decoupling)

◆ The Problem

ViewControllers handling navigation logic:

```
navigationController?.pushViewController(vc, animated: true) // ❌
```

❌ Results:

- Fat ViewControllers
- Hard to reuse screens
- Navigation tightly coupled to UI

◆ The Solution: Coordinator Pattern

Move **navigation logic** outside ViewControllers.

✅ Coordinator Protocol

```
protocol Coordinator {  
    func start()  
}
```

✅ App Coordinator Example

```
class AppCoordinator: Coordinator {  
  
    let navigationController: UINavigationController  
  
    init(navigationController: UINavigationController) {  
        self.navigationController = navigationController  
    }  
  
    func start() {  
        let vm = HomeVM(api: APIService())  
        let vc = HomeVC(viewModel: vm)  
    }  
}
```

```

        navigationController.pushViewController(vc,
animated: false)
    }
}

```

✅ ViewController (Navigation-Free)

```

class HomeVC: UIViewController {

    let viewModel: HomeVM

    init(viewModel: HomeVM) {
        self.viewModel = viewModel
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) has not been
implemented")
    }
}

```

- ✓ No navigation code
- ✓ Reusable VC
- ✓ Easy testing

🔗 DI + Coordinator = Clean Architecture

- Coordinator **injects dependencies**
- ViewControllers **focus only on UI**
- ViewModels **contain business logic**

📌 This combo is **very powerful** and highly valued in interviews.

🔑 Benefits of Coordinator Pattern

- ✓ Removes navigation from VC
- ✓ Centralized app flow
- ✓ Easy deep linking
- ✓ Scales well in large apps

Coordinator Interview One-Liner (Perfect)

“The Coordinator pattern separates navigation logic from view controllers, resulting in cleaner and more reusable screens.”

Interview Traps (Avoid These)

 “DI frameworks are mandatory”

✓ Native DI works fine

 “Coordinator replaces MVVM”

✓ Coordinator handles navigation only

 “VC should push next VC”

✓ Coordinator should

Power Interview Answer (DI + Coordinator)

“I use Dependency Injection to keep ViewModels testable and the Coordinator pattern to decouple navigation from ViewControllers, resulting in a scalable and maintainable architecture.”

6. Coordinator Pattern (Navigation Decoupling)

◆ Problem

ViewControllers handling navigation logic.

◆ Solution

Move navigation to **Coordinator**.

✅ Example

```
protocol Coordinator {  
    func start()  
}
```

◆ Benefits

- ✓ No navigation in VC
- ✓ Cleaner flow
- ✓ Reusable screens

🎯 Interview Answer

Coordinator separates navigation from view controllers.

7. Repository Pattern (Data Abstraction)

◆ What is Repository?

Single interface for data access.

◆ Flow

ViewModel → Repository → API / DB

✓ Example

```
protocol UserRepository {  
    func fetchUsers() async throws -> [User]  
}
```

◆ Benefits

- ✓ Switch API / DB easily
- ✓ Offline-first support
- ✓ Easy testing

🎯 Interview Answer

Repository abstracts data sources from business logic.

8. Architecture Comparison (Interview Gold)

Architecture	Use Case
MVC	Small apps
MVVM	Most iOS apps
VIPER	Large enterprise
Clean	Domain-heavy apps

9. Common Interview Questions

Q: Why MVVM over MVC?

A: Better testability and separation.

Q: When VIPER?

A: Large teams, complex flows.

Q: Coordinator needed?

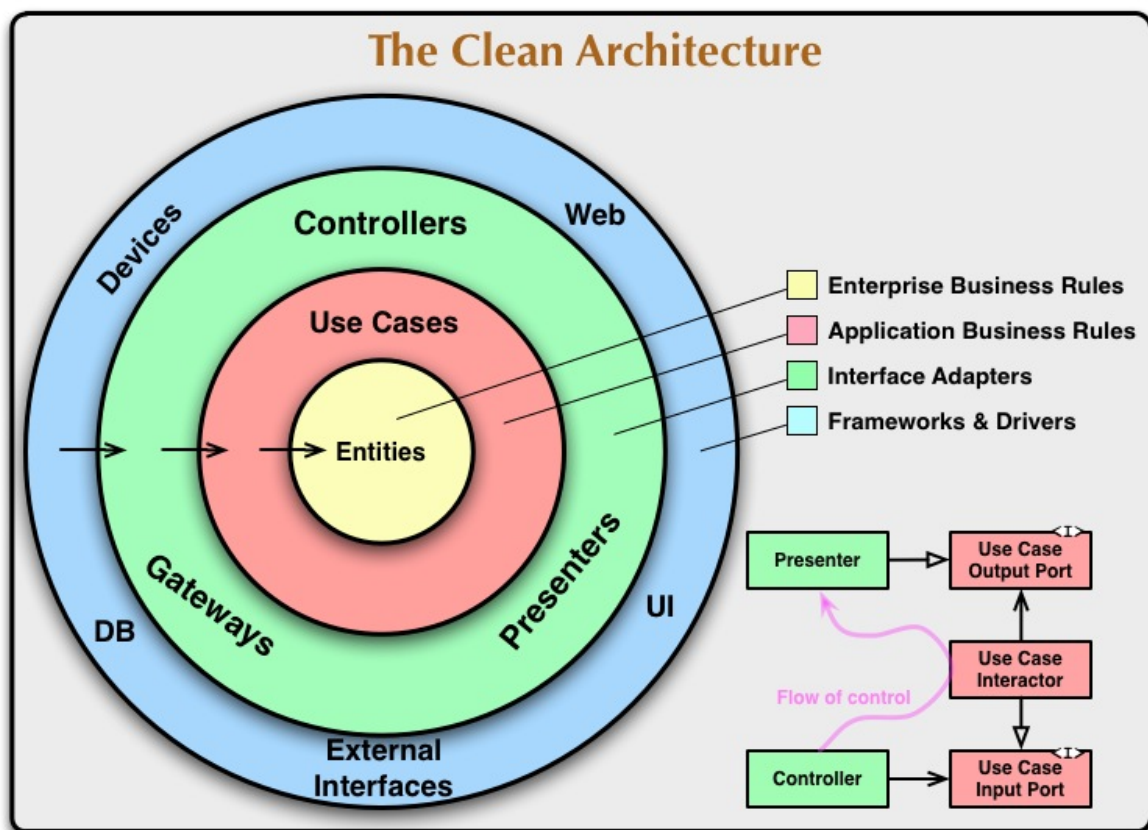
A: For complex navigation.



Power Interview Answer

I generally use MVVM with Dependency Injection, Repository pattern for data, and Coordinators for navigation. For complex domains, I follow Clean Architecture principles.

Clearn architecture



What *is* Clean Architecture?

Clean Architecture = Layered architecture that enforces:

- **Separation of concerns**
- **Independence of frameworks**
- **Testability**
- **Scalability**
- **Loose coupling**

Created by **Uncle Bob**.

★ 2. The 3-Layer Structure (MUST MEMORIZE)

PRESENTATION LAYER	→ SwiftUI / UIKit + ViewModel
DOMAIN LAYER	→ UseCases + Entities
DATA LAYER	→ Repositories + API + Persistence

★ 3. What Each Layer Contains

1 PRESENTATION LAYER (closest to UI)

Contains:

- SwiftUI Views / ViewControllers
- ViewModels (MVVM)
- State & UI logic only

Does NOT contain:

- Networking
- Core Data
- Business rules

2 DOMAIN LAYER (the heart)

Contains:

- **Entities (Models)**
- **UseCases / Interactors**
- Business rules (pure Swift, no frameworks)

This layer is:

“Framework-independent and UI-independent.”

Samsung *loves* this line.

DATA LAYER (external world)

Contains:

- Repositories (implementations)
- API calls
- Local database (Core Data, Realm)
- Network mappings

This layer is allowed to use frameworks.

4. Dependency Rule (CLEAN ARCHITECTURE'S GOLDEN RULE)

Outer layers depend on inner layers, but inner layers NEVER depend on outer layers.

Meaning:

- Presentation → depends on Domain
- Domain → depends on **nothing**
- Data → depends on Domain

Domain is the **center of the circle**.

View → ViewModel → UseCase → Repository → API/DB

↑

Entity

Explain this diagram in interviews — it wins instantly.

Entity:

```
struct User: Equatable {
    let id: Int
    let name: String
}
```

UseCase protocol:

```
protocol FetchUsersUseCase {
    func execute() async throws -> [User]
}
```

UseCase Implementation:

```
final class FetchUsersUseCaseImpl: FetchUsersUseCase {
    private let repository: UserRepository

    init(repository: UserRepository) {
        self.repository = repository
    }

    func execute() async throws -> [User] {
```

```

        try await repository.fetchUsers()
    }
}

```

2. DATA LAYER (Repository + API)

Repository Protocol (in Domain)

```

protocol UserRepository {
    func fetchUsers() async throws -> [User]
}

```

Implementation (in Data Layer)

```

final class UserRepositoryImpl: UserRepository {
    func fetchUsers() async throws -> [User] {
        let url = URL(string: "https://
jsonplaceholder.typicode.com/users")!
        let (data, _) = try await
URLSession.shared.data(from: url)
        return try JSONDecoder().decode([User].self,
from: data)
    }
}

```

3. PRESENTATION LAYER (SwiftUI + MVVM)

ViewModel:

```

@MainActor
final class UserViewModel: ObservableObject {
    @Published var users: [User] = []

    private let fetchUsersUseCase: FetchUsersUseCase

    init(fetchUsersUseCase: FetchUsersUseCase) {
        self.fetchUsersUseCase = fetchUsersUseCase
    }

    func load() async {

```

```

        do {
            users = try await fetchUsersUseCase.execute()
        } catch {
            print("Error:", error)
        }
    }
}

```

View:

```

struct UserListView: View {
    @StateObject private var vm: UserViewModel

    init() {
        let repo = UserRepositoryImpl()
        let useCase = FetchUsersUseCaseImpl(repository:
repo)
        _vm = StateObject(wrappedValue:
UserViewModel(fetchUsersUseCase: useCase))
    }

    var body: some View {
        List(vm.users, id: \.id) {
            Text($0.name)
        }
        .task { await vm.load() }
    }
}

```

7. Why Clean Architecture? (USE THIS IN INTERVIEW)

Testability

UseCases + Repository interfaces allow dependency injection.

Separation of responsibilities

UI, business logic, and data are cleanly separated.

Scalability

Large teams can work on different layers independently.

✓ Framework independence

Domain layer does not import:

- SwiftUI
- UIKit
- CoreData
- URLSession

This impresses interviewers.

★ 8. Clean Architecture + MVVM = Industry Standard

Samsung, Google, Uber, Netflix use:

MVVM + UseCases + Repositories

Because:

- ViewModel → orchestrator
- UseCase → business logic
- Repository → data access abstraction

★ 9. Common Interview Questions (with perfect answers)

? Q1: Why Clean Architecture?

Clean Architecture enforces separation of concerns, making code scalable, testable, and maintainable.

It isolates business logic from UI and data sources.

? Q2: What is the dependency rule?

Inner layers must not depend on outer layers.
Only inward dependency is allowed.

? Q3: What is the role of UseCases?

They contain the app's business logic and orchestrate data flow between ViewModel and Repository.

? Q4: What is the role of Repository?

It abstracts the data source.
UI doesn't care if data comes from API, DB, or cache.

? Q5: How does Clean Architecture help testing?

You can mock UseCases or Repository and test ViewModel independently.

★ 10. Clean Architecture vs MVVM vs VIPER

Feature	MVVM	Clean Architecture	VIPER
UI separation	Good	Excellent	Excellent
Business logic separation	OK	Excellent	Excellent
Testability	High	Very high	Very high
Complexity	Low	Medium	High
Usage	SwiftUI apps	Enterprise projects	Highly modular apps

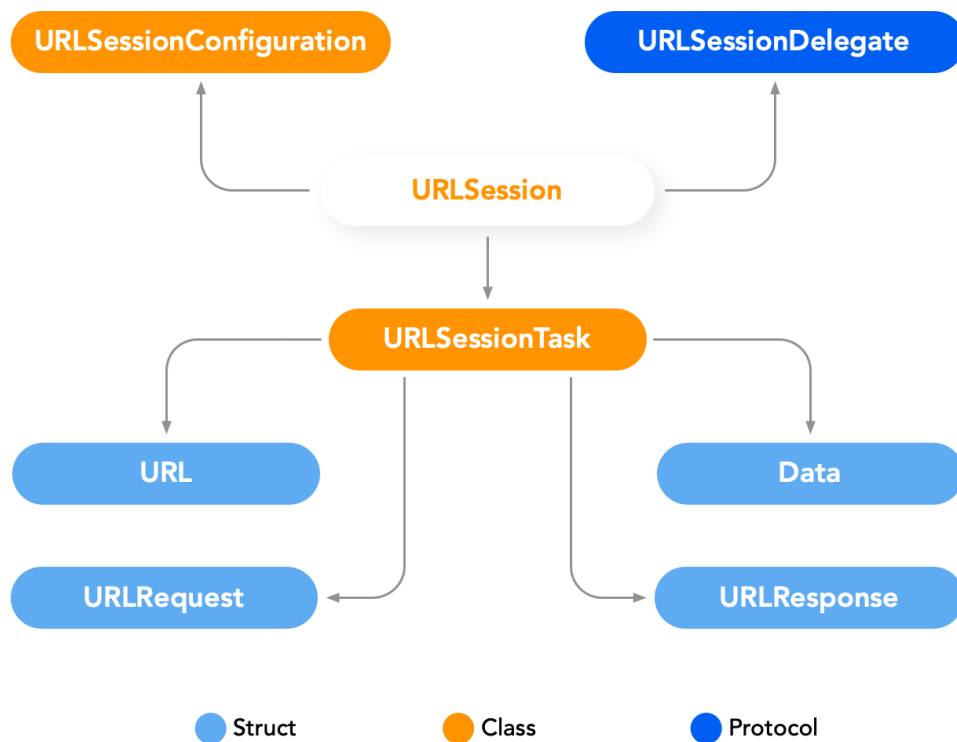
★ 11. The Best 1-Line Summary (Use this in the interview)

“Clean Architecture ensures that my UI, business logic, and data layers are independent, testable, and scalable.

ViewModels handle UI state, UseCases contain business rules, and Repositories abstract all data sources.”

5. Networking

The URLSession Constellation of Types in the iOS URL Loading System



CC BY <https://matteomanferdini.com/swift-urlsession>

Matteo
Manferdini

URLSession (Foundation of iOS Networking)

◆ What is URLSession?

`URLSession` is Apple's native API to perform:

- HTTP requests
- Downloads
- Uploads
- Background network tasks

◆ Components

Component	Role
<code>URLRequest</code>	Defines request (URL, method, headers, body)
<code>URLSession</code>	Executes requests
<code>URLSessionTask</code>	<code>DataTask</code> / <code>UploadTask</code> / <code>DownloadTask</code>
<code>URLResponse</code>	Server response

✓ Basic GET Request

```
let url = URL(string: "https://api.example.com/users")!

let task = URLSession.shared.dataTask(with: url) { data,
response, error in
    if let error = error {
        print(error)
        return
    }

    guard let data = data else { return }
    print(String(data: data, encoding: .utf8)!)
}

task.resume()
```

✓ POST Request with Body

```
var request = URLRequest(url: URL(string: "https://
api.example.com/login")!)
```

```

request.httpMethod = "POST"
request.setValue("application/json", forHTTPHeaderField:
"Content-Type")

let body = ["email": "test@test.com", "password":
"123456"]
request.httpBody = try?
JSONSerialization.data(withJSONObject: body)

URLSession.shared.dataTask(with: request).resume()

```

🎯 Interview Points

- `URLSession.shared` → simple cases
- `URLSession(configuration:)` → advanced control
- Always call `.resume()`
- Network runs on background thread automatically

2. REST APIs (How mobile apps talk to servers)

◆ REST Principles

- Stateless
- Client-server
- Resource based

◆ HTTP Methods

Method	Purpose
GET	Fetch
POST	Create
PUT	Update (full)
PATCH	Update (partial)
DELETE	Remove

◆ HTTP Status Codes

Cod e	Meaning
200	Success
201	Created
400	Bad request
401	Unauthorize d
403	Forbidden
404	Not found
500	Server error

🎯 Interview Question

Q: Difference between PUT and PATCH?

A: PUT replaces entire resource, PATCH updates partial fields.

3. JSON Parsing using Codable (MOST IMPORTANT)

◆ Why Codable?

- Type-safe
- Fast
- Cleaner than JSONSerialization

✅ Sample JSON

```
{
  "id": 1,
  "name": "Manav",
  "email": "manav@test.com"
}
```

✅ Model

```
struct User: Codable {
    let id: Int
    let name: String
    let email: String
}
```

Decode

```
let user = try JSONDecoder().decode(User.self, from:
data)
```

Snake Case → Camel Case

```
let decoder = JSONDecoder()
decoder.keyDecodingStrategy = .convertFromSnakeCase
```

Interview Points

- `CodingKeys` for custom mapping
- `decodeIfPresent` for optional fields
- `Codable` works for both encode + decode

4. Multipart Uploads (Images, Files)

Used when:

- Uploading images
- Uploading videos
- Form + file together

Multipart Body Creation

```
let boundary = UUID().uuidString
request.setValue("multipart/form-data; boundary=\
(boundary)", forHTTPHeaderField: "Content-Type")
```

```
var body = Data()
body.append("--\ (boundary)\r\n")
body.append("Content-Disposition: form-data;
name=\"image\"; filename=\"img.jpg\"\r\n")
body.append("Content-Type: image/jpeg\r\n\r\n")
body.append(imageData)
body.append("\r\n--\ (boundary)--\r\n")

request.httpBody = body
```

Interview Tip

- Multipart ≠ JSON
- Used for media uploads
- Backend expects boundary formatting

5. WebSockets (Real-time Communication)

◆ When to use?

- Chat apps
- Live tracking
- Stock prices
- Realtime notifications

✅ WebSocket using URLSession

```
let url = URL(string: "wss://example.com/socket")!
let task = URLSession.shared.webSocketTask(with: url)

task.resume()

task.receive { result in
    switch result {
    case .success(let message):
        print(message)
    case .failure(let error):
        print(error)
    }
}
```

🎯 Interview Points

- WebSocket is **bi-directional**
- Uses **wss://**
- Persistent connection

6. STOMP (Protocol over WebSocket)

◆ What is STOMP?

Simple Text Oriented Messaging Protocol

- Runs on top of WebSocket
- Used in Spring Boot backend (common)

◆ Why STOMP?

- Topic-based messaging
- Subscription model

◆ Example Flow

```
CONNECT
SUBSCRIBE /topic/chat
SEND /app/message
```

🎯 Interview Answer

STOMP provides structure and messaging semantics on top of raw WebSocket.

7. Retry Logic & Exponential Backoff

◆ Why needed?

- Network failures
- Temporary server issues

✅ Exponential Backoff Example

```
func retryRequest(attempt: Int) async throws {
    let delay = pow(2.0, Double(attempt))
```



```

        try await Task.sleep(nanoseconds: UInt64(delay *
1_000_000_000))
    }

```

◆ Best Practice

- Retry only on:
 - 500
 - Timeout
- Never retry on 401 blindly

8. Caching Strategies

◆ Types

Type	Usage
URLCache	HTTP caching
In-memory	Fast UI
Disk cache	Offline support

✓ URLCache

```

let cache = URLCache(
    memoryCapacity: 50 * 1024 * 1024,
    diskCapacity: 100 * 1024 * 1024
)
URLCache.shared = cache

```

◆ Cache Control Headers

- ETag
- Cache-Control
- If-Modified-Since

Interview Tip

Use cache for GET, never for POST.

9. OAuth / JWT Token Handling

OAuth Flow

1. Login
2. Receive access token
3. Attach token to every request
4. Refresh token when expired

Attach JWT Token

```
request.setValue("Bearer \(token)", forHTTPHeaderField:  
"Authorization")
```

Token Refresh Strategy

- Detect 401
- Call refresh API
- Retry original request

Interview Answer

JWT is stateless, encoded, signed token containing user claims.

FINAL INTERVIEW SUMMARY

If interviewer asks:

“How do you handle networking in your app?”

You answer:

I use URLSession with async/await, Codable for parsing, retry logic with exponential backoff, caching for GET requests, JWT-based authentication, WebSocket/STOMP for real-time features, and multipart uploads for media.

Data Persistence in iOS (Complete Guide)

1. UserDefaults

◆ What is it?

- Key-value storage
- Stored in plist
- Fast, lightweight

◆ Use cases

- ✓ App settings
- ✓ Flags (`isLoggedIn`)
- ✗ NOT for sensitive or large data

✓ Example

```
UserDefaults.standard.set(true, forKey: "isLoggedIn")
```

```
let loggedIn = UserDefaults.standard.bool(forKey: "isLoggedIn")
```

✗ What NOT to store

- Passwords
- Tokens
- Large JSON
- Images

🎯 Interview Answer

UserDefaults is used for small, non-sensitive configuration data.

2. Keychain (Security-Critical)

◆ What is Keychain?

- Encrypted storage provided by iOS
- Persists across app reinstall (by default)
- Protected by hardware (Secure Enclave)

◆ Use cases

- ✓ JWT tokens
- ✓ Passwords
- ✓ Refresh tokens
- ✗ UI state data

✓ Save to Keychain

```
let data = token.data(using: .utf8)!

let query: [String: Any] = [
    kSecClass as String: kSecClassGenericPassword,
    kSecAttrAccount as String: "authToken",
    kSecValueData as String: data
]

SecItemAdd(query as CFDictionary, nil)
```

🎯 Interview Points

- Safer than UserDefaults
- Used for authentication
- Requires Security framework

3. FileManager (Files & Documents)

◆ What is it?

- Manage files on disk
- Used for images, PDFs, videos, logs

◆ Directories

Directory	Usage
Documents	User-generated content
Caches	Temporary files
tmp	Auto-cleaned

✓ Save File

```
let url =
FileManager.default.urls(for: .documentDirectory,
in: .userDomainMask)[0]
let fileURL = url.appendingPathComponent("data.txt")

try "Hello".write(to: fileURL, atomically: true,
encoding: .utf8)
```

🎯 Interview Answer

FileManager is used for storing large binary or document-based data.

4. Core Data (Apple's ORM)

◆ What is Core Data?

- Object graph + persistence framework
- Not just a database
- Built on SQLite by default

◆ Core Components

Component	Role
NSPersistentContainer	Manages stack
NSManagedObjectContext	In-memory workspace
NSManagedObject	Entity
FetchRequest	Query

✓ Entity Example

```
@objc(UserEntity)
class UserEntity: NSManagedObject {
    @NSManaged var id: Int64
    @NSManaged var name: String
}
```

✓ Save Data

```
let context = persistentContainer.viewContext
let user = UserEntity(context: context)
user.id = 1
user.name = "Manav"

try context.save()
```

◆ When to use Core Data

- ✓ Complex relationships
- ✓ Offline-first apps
- ✗ Simple apps

🎯 Interview Answer

Core Data provides object graph management, relationships, caching, and persistence.

What is SwiftData?

SwiftData is Apple's **new persistence framework** (introduced in iOS 17+) that lets you store data using **pure Swift types** with **minimal boilerplate**.

One-line definition (interview-ready)

SwiftData is a Swift-native persistence framework that simplifies Core Data by using Swift macros, type safety, and seamless SwiftUI integration.



Why did Apple introduce SwiftData?

Core Data problems:

- Too much boilerplate
- `NSManagedObject`, contexts, fetch requests
- Hard for beginners
- Verbose for SwiftUI

SwiftData solves this by:

- Using **Swift structs/classes**
- Removing most Core Data APIs
- Auto-integrating with SwiftUI
- Being **compile-time safe**



Core Concept of SwiftData

Instead of this (Core Data):

```
class User: NSManagedObject {  
    @NSManaged var name: String?  
}
```

You write this (SwiftData):

```
@Model  
class User {
```



```

    var name: String
    var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }
}

```

🔥 That's it. No `NSManagedObject`, no context setup.

Key Building Blocks in SwiftData

1 @Model

Marks a class as persistable.

```

@Model
class User {
    var name: String
}

```

Equivalent to:

- Core Data → `NSManagedObject`
- Room → `@Entity`
- Realm → `Object`

2 ModelContainer

Replaces `NSPersistentContainer`

```

let container = try ModelContainer(for: User.self)

```

In SwiftUI:

```

.modelContainer(for: User.self)

```

3 ModelContext

Replaces NSManagedObjectContext

```
@Environment(\.modelContext) private var context
```

Used for:

- Insert
- Delete
- Save

@Query

Replaces NSFetchedRequest and @FetchRequest

```
@Query var users: [User]
```

Auto-updates UI when data changes 🔥

CRUD in SwiftData (VERY SIMPLE)

Create

```
let user = User(name: "Manav", age: 25)
context.insert(user)
```

Read

```
@Query var users: [User]
```

With filter:

```
@Query(filter: #Predicate<User> { $0.age > 18 })
var adults: [User]
```

Update

```
user.name = "Updated"
```

(No explicit save needed!)

Delete

```
context.delete(user)
```

SwiftData + SwiftUI = ❤️

Example View:

```
struct UserListView: View {
    @Environment(\.modelContext) private var context
    @Query var users: [User]

    var body: some View {
        List(users) { user in
            Text(user.name)
        }
    }
}
```

UI updates automatically when data changes.

Relationships in SwiftData

```
@Model
class User {
    var name: String
    var tasks: [Task] = []
}
```

```
@Model
class Task {
    var title: String
}
```

Much simpler than Core Data relationships.

SwiftData vs Core Data (IMPORTANT)

Feature	SwiftData	Core Data
Boilerplate	✓ Very low	✗ High
SwiftUI support	✓ Native	⚠ Partial
Context handling	Automatic	Manual
Type safety	Compile-time	Runtime
Learning curve	Easy	Steep
Mature	✗ New	✓ Very mature
iOS support	iOS 17+	iOS 9+

Interview line:

“SwiftData is a modern, SwiftUI-friendly abstraction on top of Core Data.”

SwiftData vs Realm vs Room (Cross-platform view)

Feature	SwiftData	Realm	Room
Platform	Apple only	Cross-platform	Android
SQL	✗ No	✗ No	✓ Yes
Live updates	✓ Yes	✓ Yes	⚠ With Flow
Threading	Easy	Easy	Medium
Apple-native	✓ Yes	✗ No	✗ No

Limitations of SwiftData (IMPORTANT for interviews)

- iOS 17+ only
- Less battle-tested
- Limited advanced features (yet)

- Not ideal for complex enterprise apps *today*
- No cross-platform support

Honest interview answer:

“SwiftData is great for new SwiftUI apps, but Core Data is still preferred for complex enterprise or legacy systems.”

5. Database Design (VERY IMPORTANT)

◆ Concepts

- Normalization
- Primary keys
- Foreign keys
- Indexing
- Relationships (1–1, 1–many, many–many)

◆ Example

User ↔ Orders

User (id PK)

Order (id PK, userId FK)

🎯 Interview Question

Q: Why indexing?

A: Faster queries on large datasets.

6. Realm (Popular 3rd-Party DB)

◆ What is Realm?

- Object-based database
- Faster than Core Data
- No context management

✅ Realm Model

```
class User: Object {
    @Persisted(primaryKey: true) var id: Int
    @Persisted var name: String
}
```

 **Save**

```
let realm = try Realm()
try realm.write {
    realm.add(User(value: ["id": 1, "name": "Manav"]))
}
```

◆ Core Data vs Realm

Core Data	Realm
Apple native	Third-party
Complex	Simple
Context-based	Thread-safe
Hard migrations	Easy

Interview Answer

Realm is easier to use and offers live-updating objects.

7. Encrypted Databases

◆ Why encryption?

- Sensitive user data
- Compliance (GDPR, PCI)

◆ Options

DB	Encryption
----	------------

Core Data	SQLCipher
Realm	Built-in encryption
SQLite	SQLCipher

✓ Realm Encryption

```
var config = Realm.Configuration()
config.encryptionKey = encryptionKey
let realm = try Realm(configuration: config)
```

🎯 Interview Answer

Encrypted databases protect data at rest against device compromise.

✓ Comparison Summary

Storage	Use
UserDefaults	Small flags
Keychain	Secure credentials
FileManager	Files
Core Data	Complex offline apps
Realm	Fast object DB
Encrypted DB	Sensitive data

🔥 Interview Power Answer

I use UserDefaults for preferences, Keychain for credentials, FileManager for files, Core Data or Realm for offline persistence, and encrypted databases for sensitive data.

Concurrency & Multithreading in iOS (Complete Guide)

1. GCD (Grand Central Dispatch)

◆ What is GCD?

Low-level C-based API for **executing tasks asynchronously** on system-managed queues.

◆ Dispatch Queues

Queue	Purpose
Main	UI updates
Global	Background work
Custom	App-specific

◆ Queue Types

- **Serial** → one task at a time
- **Concurrent** → multiple tasks simultaneously

✓ Basic Example

```
DispatchQueue.global().async {  
    // background work  
    let data = heavyTask()  
  
    DispatchQueue.main.async {  
        // UI update  
        self.label.text = data  
    }  
}
```

Interview Answer

Use GCD to perform background tasks and return to the main thread for UI updates.

2. OperationQueue (Higher-level than GCD)

◆ Why OperationQueue?

- Cancellation
- Dependencies
- Priority
- KVO

◆ Key Concepts

Concept	Meaning
Operation	Unit of work
OperationQueue	Executes operations
Dependencies	Order execution

✓ Example

```
let queue = OperationQueue()
queue.maxConcurrentOperationCount = 2

let op1 = BlockOperation {
    print("Task 1")
}

let op2 = BlockOperation {
    print("Task 2")
}

op2.addDependency(op1)
```

```
queue.addOperations([op1, op2], waitUntilFinished: false)
```

GCD vs OperationQueue

GCD	OperationQueue
Lightweight	Feature-rich
No cancellation	Supports cancellation
No dependencies	Dependencies supported

3. Thread Safety

What is Thread Safety?

Ensuring **shared data** is accessed safely by multiple threads.

Unsafe Example

```
var count = 0
```

```
DispatchQueue.concurrentPerform(iterations: 1000) { _ in  
    count += 1 //  Race condition  
}
```

Thread-safe Solution

```
let lock = NSLock()  
var count = 0
```

```
DispatchQueue.concurrentPerform(iterations: 1000) { _ in  
    lock.lock()  
    count += 1  
    lock.unlock()  
}
```

Interview Answer

Thread safety ensures consistent data access in concurrent environments.

4. Race Conditions

◆ What is a Race Condition?

Multiple threads **compete** to modify shared state, leading to unpredictable results.

✗ Example

```
balance -= 100
```

```
balance += 100
```

If run concurrently → corrupted state.

✓ Prevention

- Serial queues
- Locks
- Actors (modern)

🎯 Interview One-liner

Race condition occurs when program output depends on execution order of threads.

5. async / await (Modern Swift Concurrency)

◆ Why async/await?

- Cleaner syntax
- Structured concurrency
- Less callback hell

✓ Example

```
func fetchUsers() async throws -> [User] {
    let url = URL(string: "https://api.example.com/
users")!
    let (data, _) = try await
URLSession.shared.data(from: url)
    return try JSONDecoder().decode([User].self, from:
data)
}
```

◆ Calling

```
Task {
    let users = try await fetchUsers()
    print(users)
}
```

🎯 Interview Answer

async/await provides structured concurrency and better readability.

6. Actors (Thread Safety by Design)

◆ What is Actor?

- Reference type
- Protects mutable state automatically
- Only one task accesses state at a time

✅ Actor Example

```
actor Counter {
    private var value = 0

    func increment() {
        value += 1
    }
}
```

```

    func get() -> Int {
        value
    }
}

```

◆ Usage

```

let counter = Counter()

Task {
    await counter.increment()
    print(await counter.get())
}

```

🎯 Interview Answer

Actors eliminate race conditions by serializing access to state.

7. Task & TaskGroup

◆ Task

- Lightweight async unit
- Supports cancellation
- Structured or detached

✅ Example

```

Task {
    let result = try await fetchUsers()
}

```

◆ TaskGroup (Parallel Execution)

```

await withTaskGroup(of: Int.self) { group in
    for i in 1...3 {

```

```

        group.addTask {
            return i * 2
        }

    for await result in group {
        print(result)
    }
}

```

Interview Answer

TaskGroup runs child tasks in parallel and waits for all to complete.

8. Continuations (Bridge Old → New)

◆ Why Continuations?

Convert callback-based APIs into async/await.

✓ Example

```

func fetchData() async throws -> Data {
    try await withCheckedThrowingContinuation
    { continuation in
        legacyApiCall { data, error in
            if let error = error {
                continuation.resume(throwing: error)
            } else {
                continuation.resume(returning: data!)
            }
        }
    }
}

```

Interview Tip

- Resume **exactly once**
- Avoid memory leaks

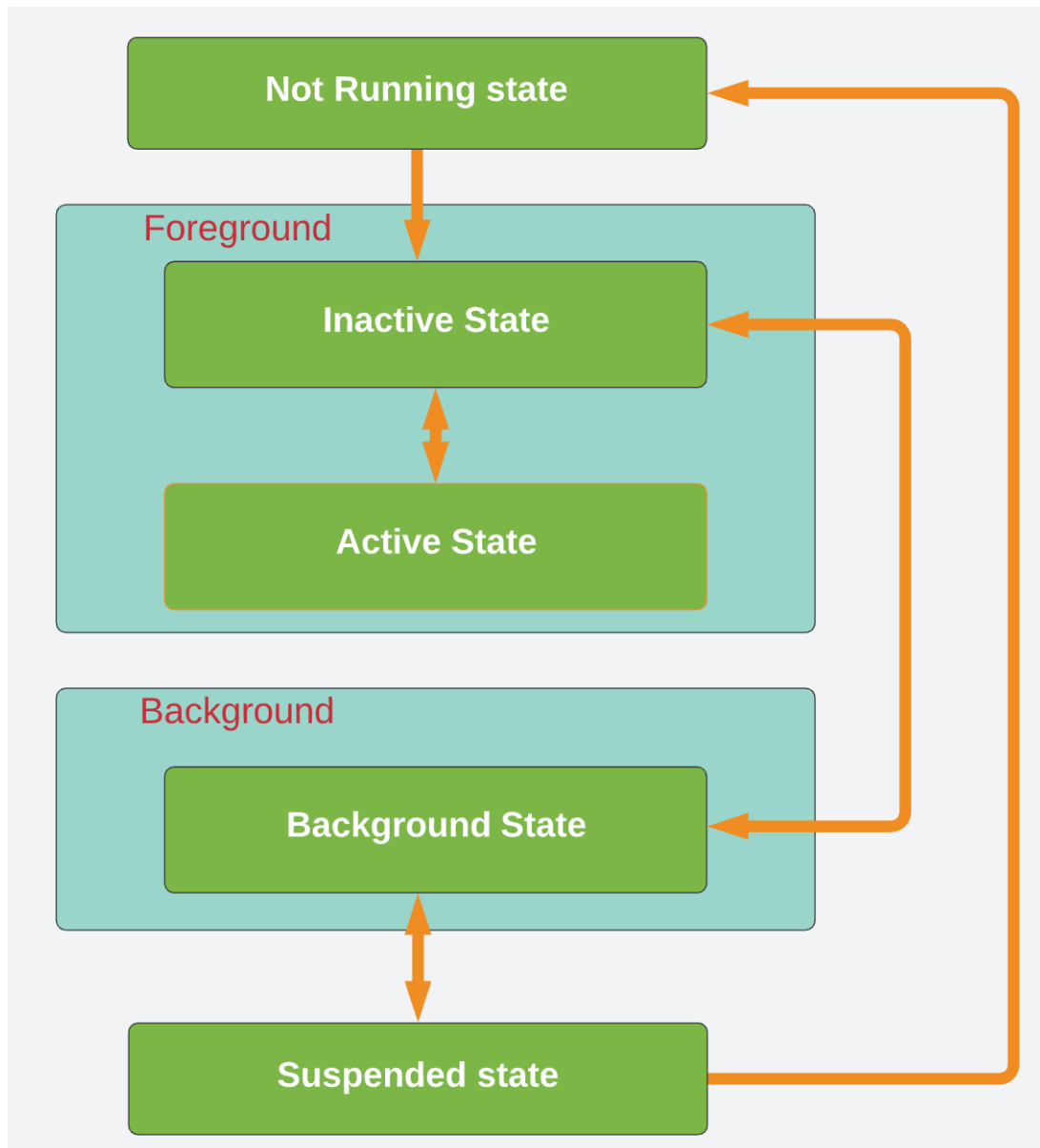
Summary Table

Topic	Use
GCD	Low-level async
OperationQueue	Dependencies
async/await	Modern concurrency
Actor	Thread-safe state
Task	Async execution
TaskGroup	Parallel tasks
Continuations	Legacy bridging

Power Interview Answer

I use async/await for networking, actors for shared state safety, TaskGroup for parallel execution, GCD for legacy code, and OperationQueue when dependencies are needed.

App State



State	Meaning
Not Running	App has not been launched or was terminated by the system/user
Inactive	Temporary state during transitions (incoming calls, notifications, app switching)
Active	App is in the foreground and receiving user input
Background	App is running code in the background (e.g., audio, location, network tasks)

Background	App is running code in the background (e.g., audio, location, network tasks)
Suspended	App is in memory but not executing any code

Typical State Flow

- **Launch:** Not Running → Inactive → Active
- **Leave App:** Active → Inactive → Background → Suspended



What happens when you close the app completely?

❌ Case 1: User kills the app from App Switcher (swipe up)

Flow:

Active → Inactive → Background → Terminated (Not Running)

◆ Important details:

- iOS **terminates the app immediately**
- App is **removed from memory**
- **No code executes after termination**
- App state becomes **Not Running**

⚠️ No lifecycle callback is guaranteed:

- `applicationWillTerminate` ❌ (usually NOT called)
- `sceneDidDisconnect` ❌ not reliable

👉 Apple does this to **protect performance and battery**

zzz Case 2: User presses Home / swipes home (normal exit)

Flow:

Active → Inactive → Background → Suspended

◆ **What this means:**

- App is still **in memory**
- App is **paused**
- iOS may **kill it later** if memory is needed
- User thinks app is “closed” — but it’s **not terminated**

 Case 3: System kills the app (low memory)

Flow:

Suspended / Background → Terminated → Not Running

◆ **Key point:**

- Happens **silently**
- User is not notified
- No termination callback

AppDelegate vs SceneDelegate

SceneDelegate was introduced in **iOS 13** mainly to support:

- **iPadOS multi-window support**
- **Multiple UI scenes per app**
- Better separation of **app logic vs UI lifecycle**



One app can now have **multiple windows (scenes)** running simultaneously.

◆ AppDelegate Responsibilities (App-Level)

Handles **global application behavior**, not UI windows.

Typical responsibilities:

- App launch configuration
- Push notification registration & handling
- Background tasks
- App-wide state changes

```
func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
    return true
}
```

 **Only one AppDelegate exists per app**

◆ SceneDelegate Responsibilities (UI-Level)

Handles **UI lifecycle and window management** for each scene.

Typical responsibilities:

- Window creation & management
- UI lifecycle transitions
- Scene-specific state handling

```
func sceneDidBecomeActive(_ scene: UIScene) {}
```

```
func sceneDidEnterBackground(_ scene: UIScene) {}
```

📌 **Multiple SceneDelegates can exist** (one per window)

🔑 Key Differences (Table)

AppDelegate	SceneDelegate
App-wide lifecycle	UI / Window lifecycle
One per app	Multiple (per scene)
Push notifications	Window management
Background tasks	Scene transitions

🎯 Interview One-Liner (Perfect Answer)

“AppDelegate manages app-level events, while SceneDelegate manages UI scenes and window lifecycle.”

★ Extra Interview Tip (Bonus)

If interviewer asks:

“Is SceneDelegate mandatory?”

- ✓ **No** — apps targeting **iOS 12 or below** don't use it
- ✓ Even on iOS 13+, you can **opt out** and manage UI in AppDelegate

App Launch (Cold Start)

Order of calls when the app is launched from Not Running state:

1 AppDelegate

`application(_:didFinishLaunchingWithOptions:)`

- ✓ App-level initialization
- ✓ SDKs, analytics, push setup

2 AppDelegate

`application(_:configurationForConnecting:options:)`

- ✓ Creates a new **UI Scene**

3 SceneDelegate

`scene(_:willConnectTo:options:)`

- ✓ Create UIWindow
- ✓ Set rootViewController

4 SceneDelegate

`sceneWillEnterForeground(_:)`

- ✓ Scene moving to foreground

5 SceneDelegate

`sceneDidBecomeActive(_:)`

- ✓ App is now interactive

App Goes to Background (Home Button / Swipe)

Order of calls:

SceneDelegate

`sceneWillResignActive(_:)`

✓ Pause ongoing tasks

SceneDelegate

`sceneDidEnterBackground(_:)`

✓ Save UI state

✓ Release shared resources

AppDelegate

`applicationDidEnterBackground(_:)`

✓ App-wide background work

work

App Returns to Foreground

Order of calls:

SceneDelegate

`sceneWillEnterForeground(_:)`

✓ Undo background changes

SceneDelegate

`sceneDidBecomeActive(_:)`

✓ Resume UI interaction

✗ App Termination (Force Kill / System Kill)

⚠ No guaranteed callbacks

- `applicationWillTerminate` ✗ unreliable
- SceneDelegate methods ✗ not guaranteed

✓ Always save state in:

`sceneDidEnterBackground(_:)`

App Launch

- └ `AppDelegate: didFinishLaunching`
- └ `AppDelegate: configurationForConnecting`
- └ `SceneDelegate: willConnectTo`
- └ `SceneDelegate: willEnterForeground`
- └ `SceneDelegate: didBecomeActive`

Background

- └ `SceneDelegate: willResignActive`
- └ `SceneDelegate: didEnterBackground`
- └ `AppDelegate: didEnterBackground`

Foreground

- └ `SceneDelegate: willEnterForeground`
- └ `SceneDelegate: didBecomeActive`

Background Modes

Background Modes allow an iOS app to **execute limited tasks when it's not in the foreground**, without draining battery or hurting system performance.

👉 iOS **strictly controls** when and how background work runs.

◆ Common Background Modes

Mode	Use Case
Audio	Music playback, podcasts
Location	GPS tracking, navigation
VOIP	Internet-based calls
Background Fetch	Periodic data sync
Remote Notifications	Silent push updates
Background Tasks	Scheduled / deferred work

◆ Background Fetch (Classic)

Allows iOS to **periodically wake your app** to fetch new content.

```
func application(
    _ application: UIApplication,
    performFetchWithCompletionHandler completionHandler:
        @escaping (UIBackgroundFetchResult) -> Void
) {
    completionHandler(.newData)
}
```

🔑 Key Points:

- Execution time is **very short**
- System decides **when** fetch happens
- Must call **completionHandler**
- Frequency depends on **user behavior & system heuristics**

◆ Modern Background Tasks (Recommended – iOS 13+)

- Uses `BGTaskScheduler`
- More reliable than Background Fetch
- Supports:
 - **BGAppRefreshTask** (short tasks)
 - **BGProcessingTask** (longer tasks, requires power)

📌 Requires:

- Task identifier in `Info.plist`
- Scheduling before app goes to background

⚠ Important Limitations (Interview Trap)

- ✗ Apps cannot run indefinitely in background
- ✗ No guaranteed execution timing
- ✗ Heavy work may be killed silently

✓ Always:

- Save state early
- Handle partial execution
- Expect interruptions

🎯 Interview One-liners (Use These)

★ Primary:

“Background execution in iOS is limited, system-controlled, and designed to preserve battery life.”

★ Alternate:

“iOS allows background work only for specific modes like audio, location, fetch, and scheduled tasks.”

Bonus Interview Q&A

Q: Which is better — Background Fetch or BGTaskScheduler?

✓ **BGTaskScheduler** (more control & reliability)

Q: Can background tasks run after force-kill?

✗ No — user force-kill disables background execution

Quick Decision Table

Use Case	Best Background Mode	Why
Music / podcast playback	Audio	Continues audio when app is backgrounded
Turn-by-turn	Location	Receives continuous GPS updates
Fitness tracking	Location	Tracks movement in background
WhatsApp / Zoom	VoIP	Maintains real-time communication
Periodic API sync	Background Fetch	Lightweight, system-scheduled
Guaranteed periodic refresh	BGAppRefreshTask	More reliable than fetch
Large uploads / DB cleanup	BGProcessingTask	Allows longer background execution
Sync after server event	Remote Notifications (silent push)	Server-driven background wake
Download when	BGProcessingTask	Runs when power/network available

How to Choose (Interview Logic)

1 Need continuous execution?

➡ Use **Audio**, **Location**, or **VoIP**

2 Need periodic but short work?

- ➡ Use **Background Fetch**
- ➡ Prefer **BGAppRefreshTask** (iOS 13+)

3 Need long-running or heavy work?

- ➡ Use **BGProcessingTask**

- Requires charging
- Requires network permission

4 Need server-triggered sync?

- ➡ Use **Silent Push Notifications**

```
{
  "aps": {
    "content-available": 1
  }
}
```

⚠ Common Interview Traps

✗ “Background fetch runs every X minutes”
✓ System decides timing

✗ “BGProcessingTask runs immediately”
✓ Runs when system conditions allow

✗ “VoIP is for any background networking”
✓ Only for call-related apps

🎯 Interview One-liners (Memorize)

★ **Best:**

“Choose background mode based on whether the task needs continuous execution, periodic refresh, or server-triggered work.”

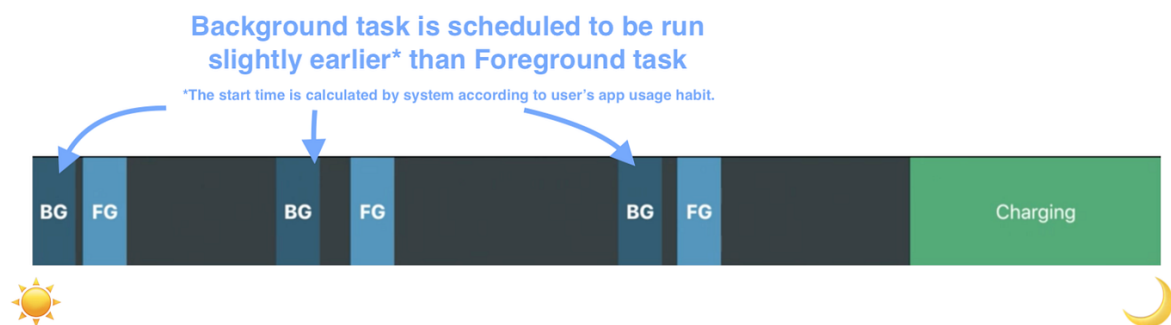
★ Alternate:

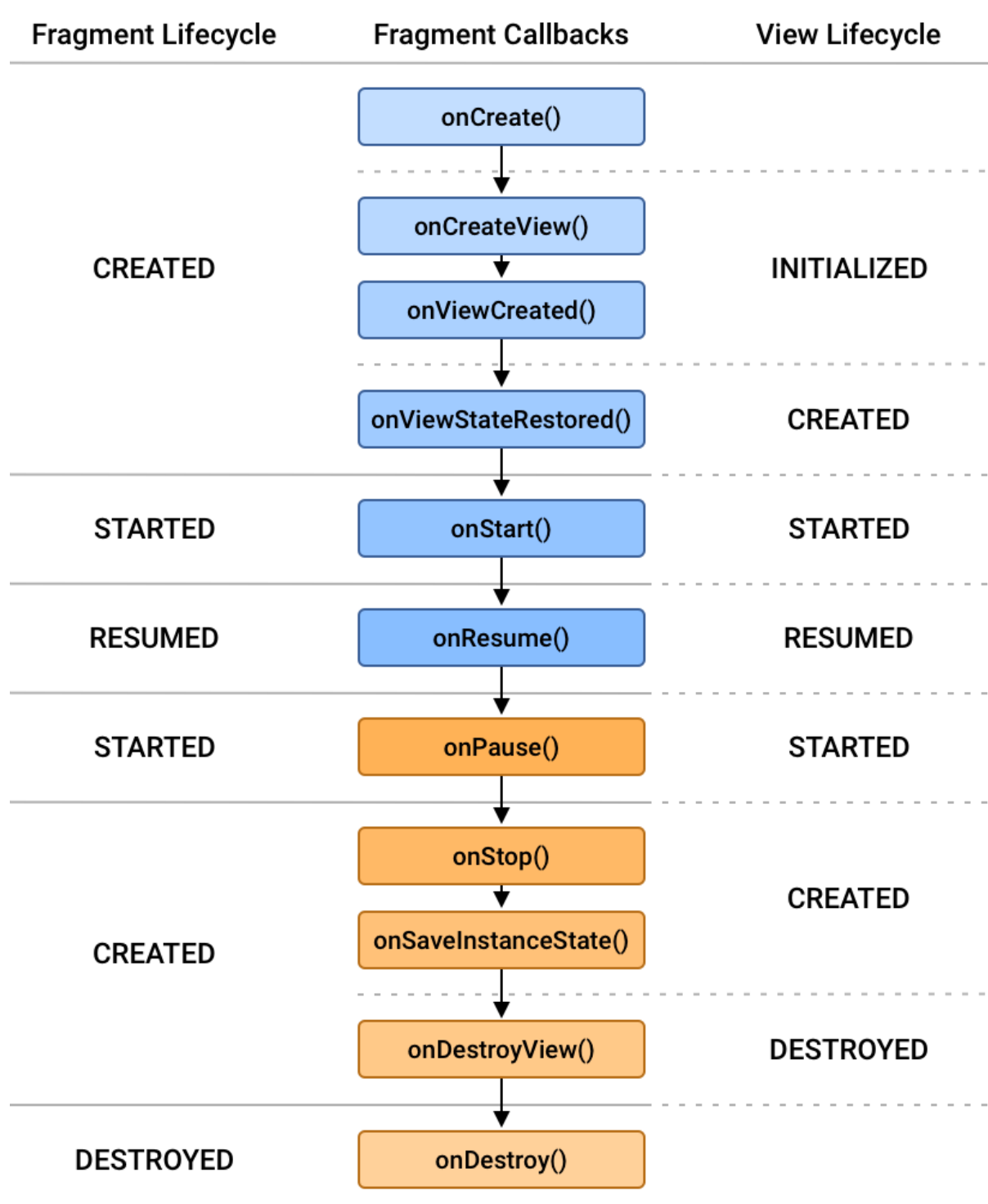
“iOS background modes are purpose-specific and tightly controlled by the system.”

🧠 Real-World Examples

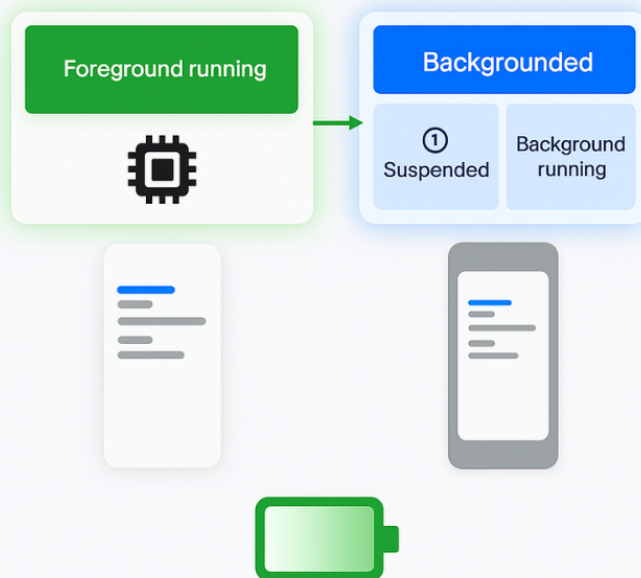
- Spotify → Audio
- Google Maps → Location
- WhatsApp → VoIP + Silent Push
- News apps → Background Fetch / BGAppRefreshTask

BGTaskScheduler Code Example (iOS 13+)





Mastering iOS 26 Background Tasks



Below is a **clean, interview-ready** example showing **registration** → **scheduling** → **handling**.

1 Add Identifier to Info.plist

```
<key>BGTaskSchedulerPermittedIdentifiers</key>
<array>
  <string>com.example.app.refresh</string>
</array>
```

2 Register the Task (AppDelegate)

Register as **early as possible** (usually in `didFinishLaunching`).

```
import BackgroundTasks

func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions:
    [UIApplication.LaunchOptionsKey: Any]?
)
```

```

) -> Bool {

    BGTaskScheduler.shared.register(
        forTaskWithIdentifier: "com.example.app.refresh",
        using: nil
    ) { task in
        self.handleAppRefresh(task: task as!
BGAppRefreshTask)
    }

    return true
}

```

3 Schedule the Task

Schedule **before app goes to background** (e.g., `sceneDidEnterBackground`).

```

func scheduleAppRefresh() {
    let request = BGAppRefreshTaskRequest(
        identifier: "com.example.app.refresh"
    )

    // Earliest time system may run it
    request.earliestBeginDate =
Date(timeIntervalSinceNow: 15 * 60)

    do {
        try BGTaskScheduler.shared.submit(request)
    } catch {
        print("Failed to schedule app refresh: \(error)")
    }
}

func sceneDidEnterBackground(_ scene: UIScene) {
    scheduleAppRefresh()
}

```

4 Handle the Background Task

This is where **actual work happens**.


```

func handleAppRefresh(task: BGAppRefreshTask) {

    // Schedule next refresh
    scheduleAppRefresh()

    let operationQueue = OperationQueue()
    operationQueue.maxConcurrentOperationCount = 1

    let operation = BlockOperation {
        // Perform lightweight network sync
        print("Background refresh running")
    }

    // Expiration handler
    task.expirationHandler = {
        operationQueue.cancelAllOperations()
    }

    operation.completionBlock = {
        task.setTaskCompleted(success: !
operation.isCancelled)
    }

    operationQueue.addOperation(operation)
}

```

BGProcessingTask (Long / Heavy Work)

Used for **uploads**, **DB cleanup**, etc.

```

let request = BGProcessingTaskRequest(identifier:
"com.example.app.processing")
request.requiresNetworkConnectivity = true
request.requiresExternalPower = true

```

```
try BGTaskScheduler.shared.submit(request)
```

📌 Requires additional **Info.plist** entries.

Key Rules (Interview Must-Know)

✓ Call `setTaskCompleted(success:)`

✓ Always **reschedule** the task

✗ No guaranteed execution time

✗ Does NOT run if user force-kills app

Interview One-liner

“BGTaskScheduler allows deferred background work that the system runs opportunistically based on power and usage patterns.”

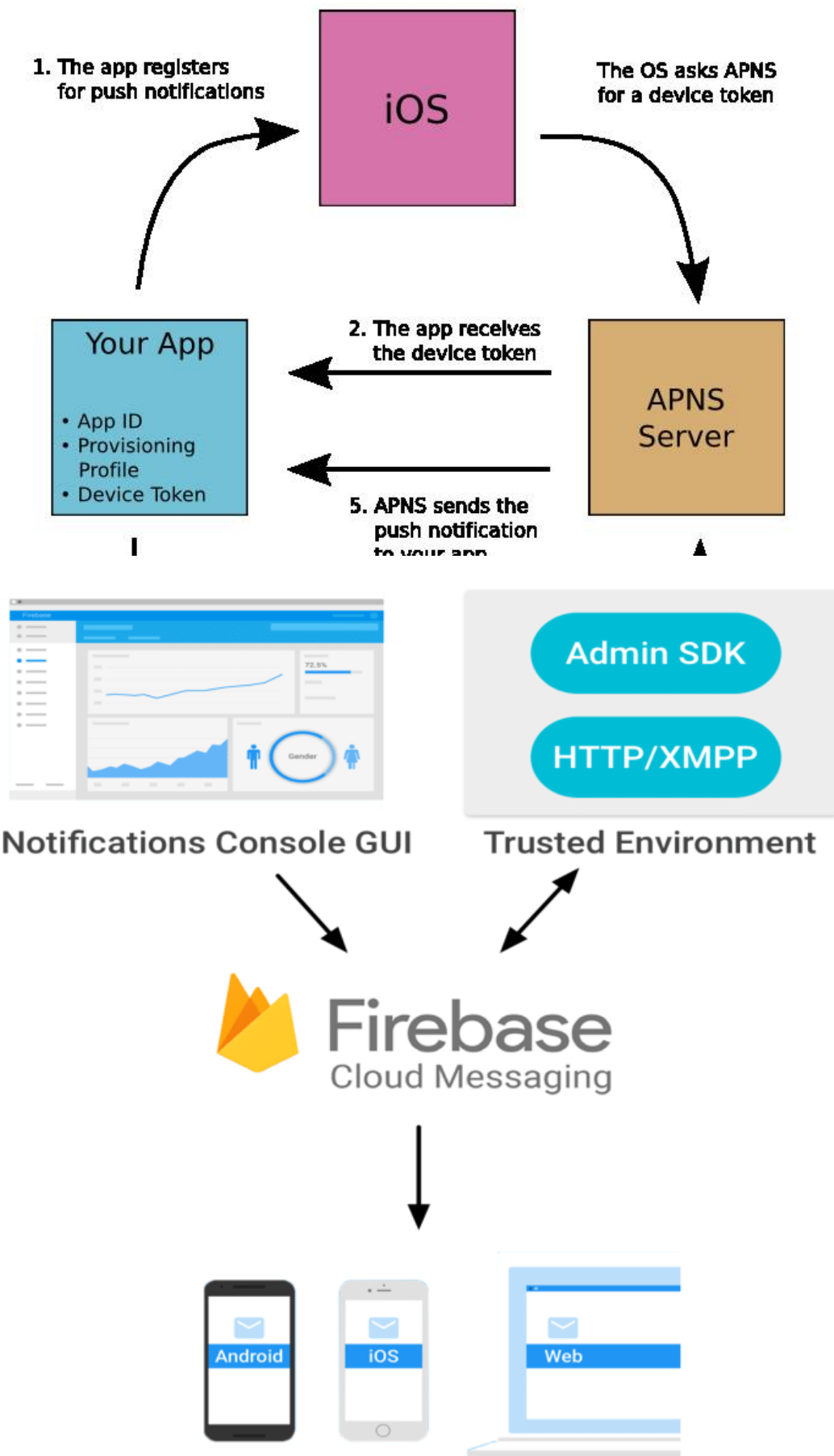
Common Interview Traps

✗ Expecting exact execution time

✗ Doing heavy work in `BGAppRefreshTask`

✓ Use `BGProcessingTask` for heavy work

Push Notifications (APNs, FCM)



- Retry & analytics
- Unified backend APIs

📌 FCM = convenience layer, APNs = **actual delivery**

◆ Push Notification Types

Type	Behavior
Alert	Displays UI (banner, sound, badge)
Silent	No UI, wakes app for background work
Rich	Images, buttons, actions

◆ Register for Push Notifications

1 Ask User Permission

```
UNUserNotificationCenter.current()
    .requestAuthorization(options:
[.alert, .badge, .sound]) { _, _ in }
```

2 Register with APNs

```
UIApplication.shared.registerForRemoteNotifications()
```

3 Receive Device Token

```
func application(
    _ application: UIApplication,
    didRegisterForRemoteNotificationsWithDeviceToken
deviceToken: Data
) {
    // Send token to backend
}
```

◆ Receiving Push Notifications

App in Foreground / Background

```
func application(
    _ application: UIApplication,
    didReceiveRemoteNotification userInfo: [AnyHashable:
Any]
) {
    // Handle notification payload
}
```

Silent Push (Background Execution)

```
{
  "aps": {
    "content-available": 1
  }
}
```

📌 Requires **Background Modes** → **Remote notifications**

🔑 APNs vs FCM (Interview Table)

APNs	FCM
Apple native service	Cross-platform layer
iOS-only	Android + iOS
Device token	FCM token
Mandatory	Optional

⚠ Interview Traps (Very Important)

❌ “FCM replaces APNs”

✓ FCM uses APNs on iOS

✗ “Silent push always wakes the app”

✓ System decides; no guarantee

✗ “Push works without user permission”

✓ Alert pushes require permission

“APNs delivers push notifications on iOS, while FCM simplifies cross-platform push handling by using APNs under the hood.”



Bonus Follow-ups Interviewers Love

- Push notification **lifecycle order**
- Difference between **silent push vs background fetch**
- Push handling with **SceneDelegate**
- Push limitations after **force-kill**

Local Notifications

What they are

Notifications **created and scheduled on the device itself** by the app.

Key points

-  **No server**
-  **No internet required**
-  Triggered by **time, calendar, or location**
-  Uses the same permission as push notifications

Examples

- Alarm / reminder
- “Drink water”
- “Your task is due”
- Fitness reminders

Simple flow

App → Schedules notification → iOS displays it

Example code (concept)

```
UNUserNotificationCenter.current().add(request)
```

Push (Remote) Notifications

What they are

Notifications **sent from a server** via **Apple Push Notification service (APNs)**.

Key points

-  **Server required**
-  **Internet required**
-  Uses **device token**
-  Can be sent anytime by backend
-  App does not control delivery time

Examples

- Chat message
- Order shipped
- Breaking news
- OTP / alerts

Simple flow

Server → APNs → Device → iOS displays notification

3 Side-by-Side Comparison (INTERVIEW GOLD)

Feature	Local Notification	Push Notification
Triggered by	App / device	Backend server
Internet needed	 No	 Yes
Server needed	 No	 Yes
Uses APNs	 No	 Yes
Uses device token	 No	 Yes
Permission required	 Yes	 Yes
Works when app closed	 Yes	 Yes

4 Common Interview Traps

✗ “Local notifications don’t need permission”

✓ **They DO need permission**

✗ “Push notifications are handled by the app”

✓ **They are handled by iOS via APNs**

✗ “Local notifications are less powerful”

✓ **They’re just different**

5 When to Use What (IMPORTANT)

Use Local Notifications when:

- Event is **predictable**
- Triggered by user action
- No backend needed
- Works offline

Use Push Notifications when:

- Event happens on server
- Real-time updates
- Multi-device sync
- Server-driven logic

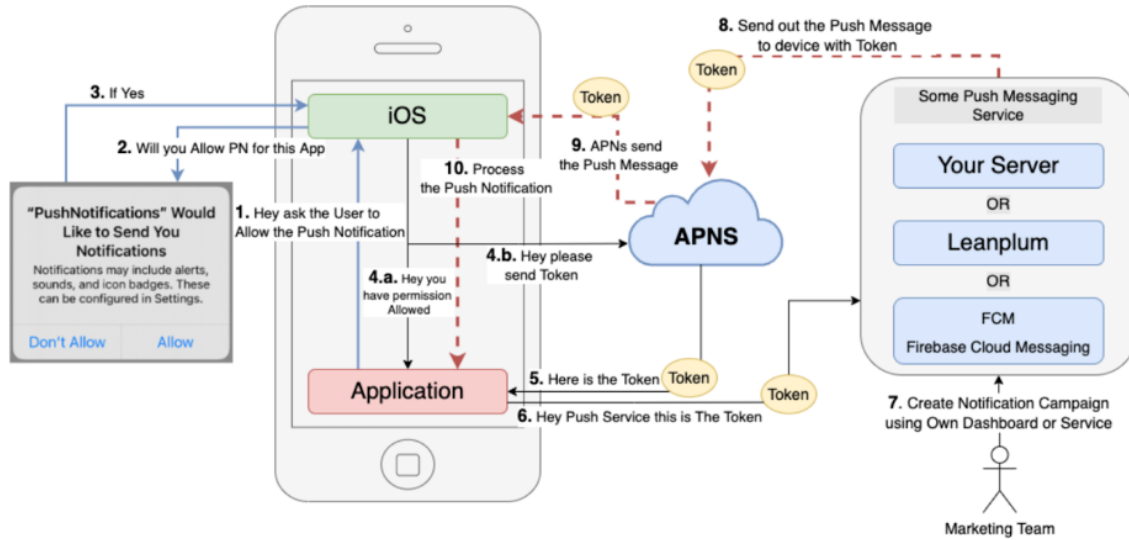
6 Samsung-Style PERFECT ANSWER (MEMORIZE)

“Local notifications are scheduled by the app on the device and don’t require a server or internet.

Push notifications are sent by a backend through APNs using a device token and require network connectivity.

Both require user permission and are displayed by the iOS system.”

Silent Push vs Normal Push (iOS)







1 Normal Push Notification

What it is

A **user-visible notification** that shows:

- Banner / alert
- Sound
- Badge

Typical uses

- Chat message
- Order shipped
- OTP / alerts
- News updates

Payload (example)

```
{
  "aps": {
    "alert": {
      "title": "New Message",
      "body": "You received a message"
    },

```

```
    "sound": "default",  
    "badge": 1  
  }  
}
```

Key points

-  **Visible to user**
-  Requires **notification permission**
-  Delivered by **APNs**
-  Cannot run background code by default

Silent Push Notification

What it is

A **background notification** that:

- Shows **NO alert**
- Plays **NO sound**
- Updates **NO badge**

Used to **wake the app in background** to do work.

Typical uses

- Background data sync
- Refresh content
- Update local database
- Prepare UI before user opens app

Payload (example)

```
{  
  "aps": {  
    "content-available": 1  
  }  
}
```

```

    }
}

```

Key points

-  Not visible to user
-  Requires **Background Modes** → **Remote notifications**
-  Can wake app briefly
-  Execution time is limited
-  Not guaranteed delivery

3 Side-by-Side Comparison (INTERVIEW GOLD)

Feature	Normal Push	Silent Push
Shows alert	 Yes	 No
Plays sound	 Yes	 No
Updates badge	 Yes	 No
Wakes app	 No (by default)	 Yes
Runs background code	 No	 Limited
User sees it	 Yes	 No
Requires permission	 Yes	 Yes
Requires Background Mode	 No	 Yes

4 App Behavior States (IMPORTANT)

When app is Foreground

- Normal push → delivered to delegate
- Silent push → background handler runs

When app is Background

- Normal push → shown by iOS
- Silent push → app wakes briefly

When app is Terminated

- Normal push → shown
- Silent push → **✗** usually ignored

5 How Silent Push Works Internally

Flow:

Server → APNs → iOS
→ App wakes in background
→ App performs small task
→ App is suspended again

Interview line:

“Silent pushes are best-effort and iOS may throttle them to save battery.”

6 Common Interview Traps **✗**

- ✗** “Silent push doesn’t need permission”
- ✓** It still requires notification permission
- ✗** “Silent push always runs”
- ✓** iOS may delay or drop it
- ✗** “Silent push works when app is killed”
- ✓** Usually no

7 When to Use What (IMPORTANT)

Use Normal Push when:

- User must be notified
- Immediate attention required
- UX is important

Use Silent Push when:

- Background sync is needed
- Data freshness matters
- No user interruption desired

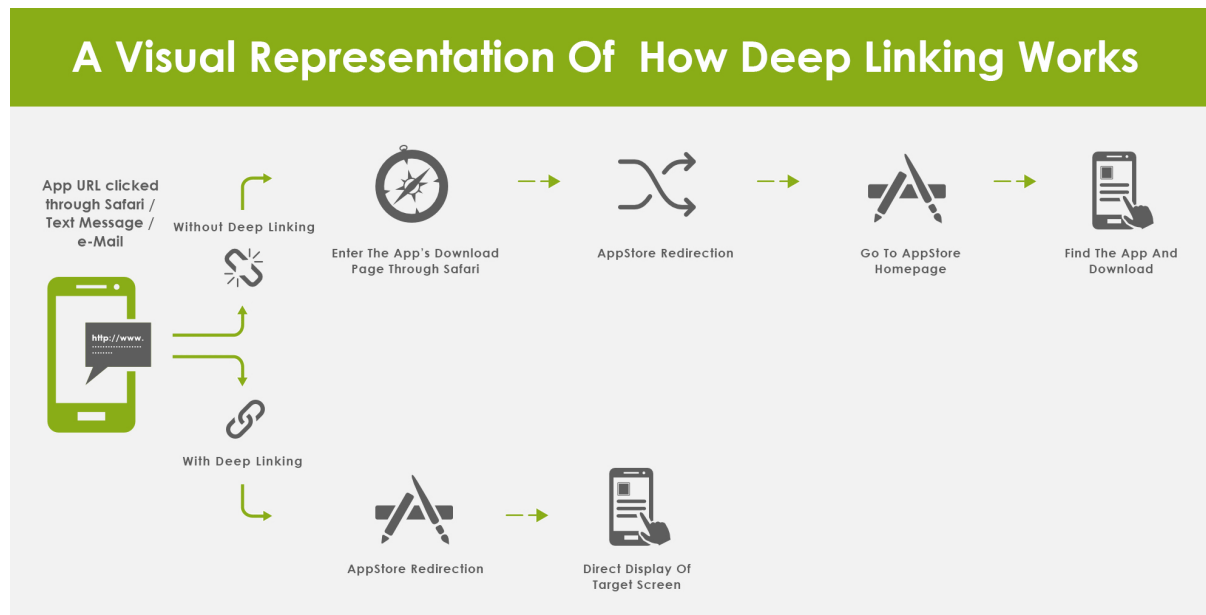
Samsung-Style PERFECT ANSWER (MEMORIZE)

“Normal push notifications are user-visible alerts delivered via APNs.

Silent push notifications contain `content-available` and are used to wake the app in background for limited processing without showing any UI.

Silent pushes are best-effort and subject to system throttling.”

Deep Links vs Universal Links



Deep Links (URL Schemes)

Format

`myapp://profile/123`

How it works

- Uses a **custom URL scheme**
- Opens the app **directly**
- Works only if the app is installed

Handling in AppDelegate

```
func application(
    _ app: UIApplication,
    open url: URL,
    options: [UIApplication.OpenURLOptionsKey : Any]
) -> Bool {
```

```
        // Parse URL and navigate
        return true
    }
}
```

⚠ Limitations

- Can **conflict** with other apps using same scheme
- Not secure (no domain ownership verification)
- If app is not installed → **nothing happens**

◆ Universal Links (Preferred by Apple)

Format

`https://myapp.com/profile/123`

How it works

- Standard **HTTPS link**
- Opens app **if installed**
- Falls back to **Safari** if not installed
- **Secure & verified** by Apple

Handling (SceneDelegate)

```
func scene(
    _ scene: UIScene,
    continue userActivity: NSUserActivity
) {
    if let url = userActivity.webpageURL {
        // Handle navigation
    }
}
```

◆ Requirements for Universal Links

1 Apple App Site Association (AASA) file

- Hosted at:

`https://myapp.com/.well-known/apple-app-site-association`

- No extension
- JSON format
- Served with `application/json`

2 Domain Verification

- Enable **Associated Domains** capability
- Add:

`applinks:myapp.com`

Deep Link vs Universal Link (Interview Table)

Deep Link	Universal Link
Custom scheme	HTTPS
Can conflict	Secure & verified
App-only	App + Web fallback
Less trusted	Apple-recommended

Interview One-Liner (Perfect)

“Deep links use custom URL schemes, while universal links use HTTPS and securely open the app if installed or Safari otherwise.”

Common Interview Traps

 “Universal links always open the app”

 User can disable them

✗ “AASA file is optional”

✓ Mandatory

✗ “Universal links work without HTTPS”

✓ HTTPS is required



When to Use What?

- **Internal testing / quick navigation** → Deep Links
- **Production apps / marketing / emails** → Universal Links ✓

App Extensions

App Extensions are **small, focused apps** that extend your app's functionality **outside the main app**, allowing users to interact with your features **without opening the full app**.

They run in **their own process** and are launched by the system when needed.

◆ Common iOS App Extensions

Extension	Use Case
Share Extension	Share content from other apps
Notification Service Extension	Modify push notification content
Widget Extension	Home Screen / Lock Screen widgets
Keyboard Extension	Custom keyboards
Siri Extension	Voice commands & shortcuts

◆ Key Rules & Constraints (Very Important)

Separate Sandbox

- Extensions **do NOT share memory** with the main app
- They run as **independent processes**

Limited Resources

- Strict **memory & execution time** limits
- Heavy work may get terminated

Communication with Main App

- Use **App Groups** to share:
 - `UserDefaults`
 - Files
 - Databases

```
let defaults = UserDefaults(suiteName:
"group.com.myapp.shared")
defaults?.set("value", forKey: "key")
```

⚠ Interview Traps (Avoid These)

✗ “Extensions can access app memory directly”

✓ They **cannot** — only via App Groups

✗ “Extensions run forever”

✓ They are **short-lived**

✗ “Extensions can do background work freely”

✓ Very limited execution

🎯 Interview One-Liner (Perfect)

“App extensions allow an app to provide functionality outside the main app while running in a separate sandbox with limited resources.”

🧠 Bonus Follow-up (Interviewers Love This)

Q: Can an extension launch the main app automatically?

✗ No — Apple restricts this

✓ User action is required

Q: Which extension is commonly used to modify push notifications?

✓ Notification Service Extension

1 Share Extension (Most Common)

✓ Use case

Share text, URLs, or images from **Safari / Photos / any app** into your app

👉 Example: *Save link to notes app*

How it works

- Appears in iOS **Share Sheet**
- Receives data via `NSExtensionItem`

Example Code

```
class ShareViewController: UIViewController {  
  
    override func viewDidLoad() {  
        super.viewDidLoad()  
  
        let item = extensionContext?.inputItems.first as?  
        NSExtensionItem  
        let provider = item?.attachments?.first  
  
        if  
provider?.hasItemConformingToTypeIdentifier("public.url")  
== true {  
            provider?.loadItem(forTypeIdentifier:  
"public.url", options: nil) { item, _ in  
                print("Shared URL:", item ?? "")  
            }  
        }  
    }  
}
```

Interview Point

Share Extensions let users send content to your app without opening it.

2 Notification Service Extension

Use case

Modify push notification **before it is shown**

- Download image
- Change title/body

- Decrypt payload

Example Code

```
class NotificationService: UNNotificationServiceExtension
{
    override func didReceive(
        _ request: UNNotificationRequest,
        withContentHandler contentHandler:
            @escaping (UNNotificationContent) -> Void
    ) {
        let content = request.content.mutableCopy() as!
        UNMutableNotificationContent
        content.title = "Modified Title 🚀"

        contentHandler(content)
    }
}
```

Interview Point

Runs briefly to customize notification UI.

3 Widget Extension (WidgetKit)

Use case

Home Screen / Lock Screen widgets

👉 Example: Weather, steps count, stocks

Example Code

```
struct SimpleWidget: Widget {
    var body: some WidgetConfiguration {
        StaticConfiguration(
            kind: "SimpleWidget",
            provider: Provider()
        )
    }
}
```

```

        ) { entry in
            Text("Hello Widget 🙌")
        }
    }
}

```

🔑 Interview Point

Widgets are read-only UI powered by timelines.

4 Custom Keyboard Extension

✅ Use case

Third-party keyboards (emoji, language input)

🧩 Example Code

```

class KeyboardViewController: UIInputViewController {

    @IBAction func keyPressed(_ sender: UIButton) {
        textDocumentProxy.insertText("😊")
    }
}

```

🔒 Privacy Rule

- Full access required for network usage

5 Siri Extension (Intents)

✅ Use case

Voice commands via Siri

👉 “Hey Siri, add a task”

🧩 Example

```
class IntentHandler: INExtension {
    override fun handler(for intent: INIntent) -> Any {
        return self
    }
}
```

Sharing Data with Main App (App Groups)

```
let sharedDefaults =
    UserDefaults(suiteName: "group.com.myapp.shared")

sharedDefaults?.set("Hello", forKey: "message")
```

 Required because extensions have **separate sandbox**

Key Rules (Interview Must-Know)

Rule	Meaning
Separate sandbox	No shared memory
Limited execution	Can be killed anytime
App Groups	Only way to share data

Perfect Interview One-Liner

“App Extensions provide focused functionality outside the main app while running in a separate sandbox with strict system limits.”

Widgetkit



Widgets are **glanceable, lightweight UI components** shown on the **Home Screen / Lock Screen**.

Key characteristics:

- **✗** No direct user interaction (no buttons, gestures)
- **✓** Tap → opens the app
- **🕒** **Timeline-based updates** (not real-time)
- **⚡** Designed for low battery usage

◆ WidgetKit Core Components

Component	Role
TimelineProvider	Supplies data & refresh schedule
TimelineEntry	Snapshot of data at a point in time
Widget View	SwiftUI UI for the widget

◆ Timeline Flow (How Widgets Update)

`Timeline(entries: entries, policy: .after(nextUpdate))`

What this means:

- Widget renders entries **in sequence**
- System decides **when to reload**
- `.after(nextUpdate)` requests the next refresh time

📌 iOS may delay updates to save battery

◆ Minimal Widget Example

```
struct SimpleEntry: TimelineEntry {
    let date: Date
}

struct Provider: TimelineProvider {

    func placeholder(in context: Context) -> SimpleEntry
    {
        SimpleEntry(date: Date())
    }

    func getSnapshot(
        in context: Context,
        completion: @escaping (SimpleEntry) -> Void
    ) {
        completion(SimpleEntry(date: Date()))
    }
}
```

```

    ) {
        completion(SimpleEntry(date: Date()))
    }

    func getTimeline(
        in context: Context,
        completion: @escaping (Timeline<SimpleEntry>) ->
Void
    ) {
        let entry = SimpleEntry(date: Date())
        let nextUpdate = Calendar.current
            .date(byAdding: .minute, value: 30, to:
Date())!

        completion(
            Timeline(entries: [entry],
policy: .after(nextUpdate))
        )
    }
}

```

◆ Data Sharing with Main App

Widgets run in a **separate sandbox**, so they must use **App Groups**.

```

let sharedDefaults =
    UserDefaults(suiteName: "group.com.myapp.shared")

let value = sharedDefaults?.string(forKey: "key")

```

⚠ Widget Limitations (Interview Traps)

- ✗ No real-time updates
- ✗ No background tasks inside widget
- ✗ No heavy networking

✓ Use:

- App → save data
- Widget → read snapshot

Interview One-Liner (Perfect)

“Widgets are lightweight, timeline-driven SwiftUI views designed for glanceable information, not real-time interaction.”

Common Follow-up Questions

Q: Can widgets update themselves?

 No — system-controlled

Q: How do widgets get fresh data?

 App Groups + timeline reload

Q: Can widgets run network calls?

 Very limited, not recommended

Memory Management

ARC (Automatic Reference Counting) is a compiler-based memory management system used in Swift (and Objective-C).

Key points:

- Automatically inserts **retain** / **release** calls
- Determines object lifetime at **compile time**
-  Not a runtime garbage collector
- Frees memory when **reference count reaches zero**

◆ How ARC Works (Step-by-Step)

```
class Person {
    let name: String
    init(name: String) {
        self.name = name
    }
    deinit {
        print("\(name) deinitialized")
    }
}

var p1: Person? = Person(name: "Manav") // retain count =
1
var p2 = p1 // retain count =
2

p1 = nil // retain count =
1
p2 = nil // retain count =
0 → deinit
```

 When **retain count = 0**, ARC automatically calls `deinit`.

◆ Strong, Weak & Unowned References

Reference	Retains Object	When to Use
strong (default)	✓ Yes	Ownership
weak	✗ No	Delegate, parent-child
unowned	✗ No	Guaranteed non-nil

◆ Retain Cycle (Memory Leak)

✗ Problem

```
class A {
    var b: B?
}
```

```
class B {
    var a: A?
}
```

Both objects keep each other alive → **leak**

✓ Fix (Weak Reference)

```
class B {
    weak var a: A?
}
```

◆ Closures & ARC (Very Important)

✗ Retain Cycle

```
class ViewModel {
    var onUpdate: (() -> Void)?

    func setup() {
        onUpdate = {
            self.refresh()
        }
    }
}
```

```

    }
}

```

✅ **Fix**

```

onUpdate = { [weak self] in
    self?.refresh()
}

```

⚠️ **Common Interview Traps**

- ❌ “ARC is garbage collection”
- ✅ Compile-time reference counting
- ❌ “Weak can be used anywhere”
- ✅ Only for **class** references
- ❌ “deinit always called”
- ✅ Not if retain cycle exists

🎯 **Interview One-Liner (Perfect)**

“ARC automatically manages object lifetimes by tracking strong references and releasing objects when their reference count reaches zero.”

Strong / Weak / Unowned

◆ Strong (Default)

- Increases retain count
- Keeps object alive

```
var person: Person?
```

◆ Weak

- Does NOT increase retain count
- Automatically becomes `nil`
- Must be optional

```
weak var delegate: MyDelegate?
```

◆ Unowned

- Does NOT increase retain count
- Never becomes `nil`
- Crashes if object deallocated

```
unowned let owner: Owner
```

◆ Comparison Table

Type	Retain s	Optiona l	Crash Risk
strong	✓	✗	✗
weak	✗	✓	✗
unowne d	✗	✗	✓

🎯 Interview Rule

Use `weak` when reference can become `nil`, `unowned` when lifetime is guaranteed.

Retain Cycles (Most Common Bug)

◆ What is Retain Cycle?

Two or more objects **hold strong references to each other**, preventing deallocation.

```
class Parent {
    var child: Child?
}

class Child {
    var parent: Parent?
}
```

✓ Fix

```
class Child {
    weak var parent: Parent?
}
```

◆ Closure Retain Cycle

```
class ViewModel {
    var onUpdate: (() -> Void)?

    func load() {
        onUpdate = {
            self.refresh() // ✗ strong capture
        }
    }
}
```

✓ Fix with Capture List

```
onUpdate = { [weak self] in
    self?.refresh()
}
```

🎯 Interview Answer

Retain cycles commonly occur between closures, delegates, and parent-child references.

Memory Leaks

◆ What is a Memory Leak?

Object remains in memory **even when not needed**.

◆ Common Causes

- Strong reference cycles
- Closures capturing `self`
- Timers not invalidated
- NotificationCenter observers not removed
- Static references

✗ Timer Leak

```
Timer.scheduledTimer(withTimeInterval: 1, repeats: true)
{ _ in
    self.update()
}
```

✓ Fix

```
Timer.scheduledTimer(withTimeInterval: 1, repeats: true)
{ [weak self] _ in
    self?.update()
}
```

🎯 Interview Question

Q: Why NotificationCenter causes leaks?

A: Observers are strongly retained if not removed.

Instruments (Debugging Memory Issues)

◆ Leaks Instrument

- Detects leaked objects
- Shows retain cycles

Steps:

1. Xcode → Product → Profile
2. Choose **Leaks**
3. Interact with app
4. Analyze leaked instances

◆ Allocations Instrument

- Tracks memory allocation
- Identify objects not released

◆ Time Profiler

- CPU performance
- Identify heavy methods

🎯 Interview Answer

I use Instruments to track leaks, allocations, and CPU bottlenecks.

6. Practical Debugging Tips

◆ `deinit` Check

```
deinit {  
    print("Deallocated")  
}
```

◆ Xcode Memory Graph

- Debug → View Memory Graph
- Visual retain cycle detection

7. Interview Traps & Edge Cases

? Is ARC same as Garbage Collection?

✗ No. ARC is compile-time, GC is runtime.

? Does ARC work for structs?

✗ No, value types use stack/reference semantics.

? Can ARC handle circular references?

✗ Only if weak/unowned used properly.

🔥 Power Interview Answer

I rely on ARC for memory management, use weak/unowned to avoid retain cycles, carefully handle closures and delegates, and debug leaks using Instruments and Xcode's Memory Graph.

Performance Optimization

Instruments

◆ What is Instruments?

Apple's profiling suite to analyze:

- CPU usage
- Memory usage
- GPU rendering
- Leaks

◆ Most Used Instruments

Instrument	Purpose
Time Profiler	CPU bottlenecks
Allocations	Memory usage
Leaks	Memory leaks
Core Animation	GPU rendering
Network	Network delays

🎯 Interview Answer

I use Instruments to identify performance bottlenecks before optimizing.

CPU Performance Optimization

◆ Common CPU Issues

- ✗ Heavy work on main thread
- ✗ Large loops
- ✗ JSON parsing on UI thread
- ✗ Layout recalculations

✗ Bad Example

```
for _ in 0..<1_000_000 {  
    print("Heavy work")  
}
```

✓ Fix

```
DispatchQueue.global(qos: .userInitiated).async {  
    heavyWork()  
}
```

◆ Time Profiler

- Shows call stack
- Identify slow functions

🎯 Interview Tip

Keep main thread free for UI work.

GPU Performance (Rendering & Animations)

What the GPU Handles

On iOS, the **GPU** is responsible for:

- Drawing & compositing layers
- Animations (Core Animation)
- Transparency (alpha blending)
- Shadows, corner radius, masks

📌 Poor GPU usage → **dropped frames & janky animations**

◆ Common GPU Performance Problems

- ✗ **Offscreen rendering**
- ✗ Too many overlapping layers
- ✗ Excessive transparency (alpha < 1.0)
- ✗ Shadows without rasterization or shadow path

✗ Expensive Shadow (Triggers Offscreen Rendering)

```
view.layer.shadowOpacity = 0.5
```

🔴 Causes dynamic shadow calculation every frame

✅ Optimized Shadow (GPU-Friendly)

```
view.layer.shadowPath =  
    UIBezierPath(rect: view.bounds).cgPath
```

- ✓ Predefines shadow shape
- ✓ Avoids offscreen rendering
- ✓ Much faster rendering

◆ Other Common GPU Pitfalls & Fixes

1 Transparency (Alpha Blending)

- ✗ Multiple semi-transparent views stacked
- ✓ Set `backgroundColor` opaque where possible

2 Corner Radius + Masks



- ```
view.layer.cornerRadius = 12
view.layer.masksToBounds = true
```
- ✓ Pre-render using images or avoid masks in scrolling views

### 3 Rasterization (Use Carefully)

- ```
view.layer.shouldRasterize = true
view.layer.rasterizationScale = UIScreen.main.scale
```
- ✓ Good for static views
 - ✗ Bad for animated views

◆ Core Animation Instrument (Must Know)

Color Blended Layers

- **Red** → expensive alpha blending
- Goal: minimize red areas

Color Offscreen Rendered

- Yellow highlights offscreen rendering
- Fix by removing masks, shadows, corner radius

 Xcode → Debug → View Debugging

Interview Traps

 “CPU profiling is enough for animations”

 GPU profiling is equally important

 “Rasterization always improves performance”

 Only for static content

Interview One-Liner (Perfect)

“GPU bottlenecks in iOS mainly come from overdraw and offscreen rendering caused by transparency, shadows, and complex layers.”

Real-World Example

- Laggy table scroll → rounded cells with shadows
- Fix → remove shadow or pre-render image

Memory Optimization

◆ Common Memory Issues

- ✗ Large images
- ✗ Retain cycles
- ✗ Too many cached objects

◆ Allocations Instrument

- Track object lifetime
- Find memory spikes

◆ Best Practices

- ✓ Release unused objects
- ✓ Use autoreleasepool for loops
- ✓ Avoid strong reference cycles

🎯 Interview Tip

Memory pressure can cause app termination.

Image Optimization (VERY IMPORTANT)

◆ Image Problems

- ✗ Loading full-resolution images
- ✗ Decoding on main thread
- ✗ Re-downloading images

✗ Bad

```
imageView.image = UIImage(named: "huge_image")
```

✓ Resize Before Display

```
UIGraphicsBeginImageContextWithOptions(size, true, 0)
image.draw(in: rect)
let resized = UIGraphicsGetImageFromCurrentImageContext()
UIGraphicsEndImageContext()
```

◆ Use Caching

- NSCache
- URLCache
- SDWebImage / Kingfisher

🎯 Interview Answer

Always resize images to display size and cache them.

Lazy Loading

◆ What is Lazy Loading?

Load data **only when needed**, not upfront.

✅ Swift Lazy Property

```
lazy var expensiveObject = ExpensiveObject()
```

◆ TableView Lazy Loading

- Pagination
- Load images on demand

🎯 Interview Answer

Lazy loading improves startup time and memory usage.

7. Smooth Scrolling Optimization (UITableView / UICollectionView)

◆ Common Scrolling Issues

- ✗ Image loading in cell
- ✗ Heavy layout in `cellForRow`
- ✗ Dynamic height miscalculations

✗ Bad Cell

```
func tableView(_ tableView: UITableView, cellForRowAt  
indexPath: IndexPath) -> UITableViewCell {
```

```
heavyComputation()  
cell.imageView?.image = loadImage()  
}
```

Optimized Cell

```
cell.imageView?.image = placeholder  
loadImageAsync { image in  
    cell.imageView?.image = image  
}
```

Best Practices

- ✓ Cell reuse
- ✓ Avoid AutoLayout complexity
- ✓ Use `prepareForReuse()`
- ✓ Prefetching APIs

Interview Answer

Smooth scrolling requires lightweight cells and async image loading.

“loadImageAsync represents asynchronous image loading on a background thread to keep the main thread free for smooth scrolling.”

Security

1. Keychain (Secure Credential Storage)

◆ What is Keychain?

- Encrypted storage managed by iOS
- Backed by **Secure Enclave**
- Persists across app reinstalls (by default)

◆ What to store

- ✓ Access tokens
- ✓ Refresh tokens
- ✓ Passwords
- ✗ UI state
- ✗ App preferences

✓ Save to Keychain

```
let data = token.data(using: .utf8)!

let query: [String: Any] = [
    kSecClass as String: kSecClassGenericPassword,
    kSecAttrAccount as String: "authToken",
    kSecValueData as String: data
]

SecItemAdd(query as CFDictionary, nil)
```

✓ Read from Keychain

```
let query: [String: Any] = [
    kSecClass as String: kSecClassGenericPassword,
```

```

        kSecAttrAccount as String: "authToken",
        kSecReturnData as String: true
    ]

    var item: AnyObject?
    SecItemCopyMatching(query as CFDictionary, &item)

```

Interview Answer

Keychain is used for storing sensitive data securely, unlike UserDefaults.

2. Face ID / Touch ID (Biometric Authentication)

Framework

LocalAuthentication

When to use

-  App unlock
-  Payment confirmation
-  Viewing sensitive data

FaceID / TouchID Example

```

import LocalAuthentication

let context = LAContext()
var error: NSError?

if
context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error: &error) {

context.evaluatePolicy(.deviceOwnerAuthenticationWithBiometrics,

```

```

                                localizedReason:
"Authenticate") { success, _ in
    if success {
        print("Authenticated")
    }
}
}

```

◆ Fallback

- Passcode automatically used if biometrics fail

🎯 Interview Answer

Biometrics provide secure, hardware-backed authentication without exposing credentials.

3. SSL Pinning (Prevent MITM Attacks)

◆ What is SSL Pinning?

Validating **server certificate or public key** inside the app.

◆ Why?

- ✗ Prevents Man-in-the-Middle attacks
- ✗ Stops fake certificates

◆ Types

Type	Description
Certificate pinning	Match full cert
Public key pinning	Match key only (preferred)

✅ URLSession SSL Pinning

```
func urlSession(_ session: URLSession,
                didReceive challenge:
URLAuthenticationChallenge,
                completionHandler: @escaping
(URLSession.AuthChallengeDisposition, URLCredential?) ->
Void) {

    if let serverTrust =
challenge.protectionSpace.serverTrust {
        completionHandler(.useCredential,
URLCredential(trust: serverTrust))
    }
}
(In real apps, compare cert/public key)
```

Interview Answer

SSL pinning ensures the app communicates only with trusted servers.

4. Secure Local Storage (Data at Rest)

◆ What does “secure storage” mean?

Protecting data **even if device is compromised**.

◆ Best Practices

- ✓ Use Keychain for credentials
- ✓ Encrypt databases
- ✓ Avoid storing secrets in plaintext
- ✓ Enable App Transport Security (ATS)

◆ Core Data Encryption

- Use SQLCipher
- Encrypt SQLite file

◆ Realm Encryption

```
var config = Realm.Configuration()  
config.encryptionKey = encryptionKey  
let realm = try Realm(configuration: config)
```

🎯 Interview Answer

Sensitive data must be encrypted at rest and in transit.

5. Hashing vs Encryption (VERY IMPORTANT)

◆ Hashing

- One-way
- Cannot be reversed
- Used for passwords, integrity

Examples:

- SHA-256
- SHA-512

◆ Encryption

- Two-way
- Can be decrypted with key
- Used for data storage & transmission

Examples:

- AES
- RSA

◆ Comparison

Feature	Hashing	Encryption
Reversible	✗	✓
Use case	Passwords	Data storage
Key needed	✗	✓

✓ Hash Example (SHA-256)

```
import CryptoKit

let hash = SHA256.hash(data: Data("password".utf8))
```

🎯 Interview Answer

Passwords should be hashed, not encrypted.

6. Common Security Mistakes (Interview Traps)

- ✗ Storing tokens in UserDefaults
- ✗ Hardcoding API keys
- ✗ Logging sensitive data
- ✗ No SSL pinning for critical apps
- ✗ Insecure deep links

7. App Transport Security (ATS)

- Enforces HTTPS
- Blocks insecure connections by default

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <false/>
```

</dict>

8. Security Interview Rapid-Fire

Q: Where do you store JWT?

A: Keychain.

Q: Why not UserDefaults?

A: Not encrypted.

Q: FaceID fallback?

A: Device passcode.

Q: Hash or encrypt password?

A: Hash.



Power Interview Answer

I use Keychain for credentials, biometrics for authentication, SSL pinning to prevent MITM attacks, encrypted databases for sensitive data, and hashing for passwords.

iOS System Frameworks – Complete Guide

1. Combine (Reactive Programming)

Combine is a **reactive programming framework** by Apple.
It handles **asynchronous data streams** using:

- Publishers
- Subscribers
- Operators

Combine works beautifully with **MVVM**, **SwiftUI**, and **URLSession**.

★ 2. Core Concepts (VERY Important)

✓ Publisher

Produces values over time (like a stream).

✓ Subscriber

Listens to values from a publisher.

✓ Operator

Transforms values in the stream (map, filter, debounce, etc.)

★ 3. Example of a Publisher

```
let publisher = Just("Hello")
```

This publisher sends **one value** then completes.

Subscribe:

```
let cancellable = publisher.sink { value in  
    print(value)  
}
```


Output:

Hello

★ 4. @Published — THE MOST IMPORTANT PROPERTY WRAPPER

Inside ViewModel:

```
class CounterViewModel: ObservableObject {  
    @Published var count = 0  
}
```

What @Published does:

- Makes `count` a **publisher**
- Any subscriber will receive updates automatically

SwiftUI watches @Published values automatically.

★ 5. Observing @Published Manually

```
vm.$count.sink { newValue in  
    print("Count is now \(newValue)")  
}
```

Note:

- `vm.$count` → publisher for the property
- `.sink` → subscriber

★ 6. Combine in MVVM (MOST IMPORTANT)

ViewModel:

```

class LoginViewModel: ObservableObject {
    @Published var username = ""
    @Published var isValid = false

    private var cancellables = Set<AnyCancellable>()

    init() {
        $username
            .map { $0.count > 3 }
            .assign(to: \.isValid, on: self)
            .store(in: &cancellables)
    }
}

```

Explanation:

1. `$username` → publisher
2. `.map` → operator (transform username to boolean)
3. `.assign` → update another `@Published` property
4. `.store` → keep subscription alive

★ 7. Combine Operators (Interview GOLD)

Operator	Meaning
<code>map</code>	transform values
<code>filter</code>	only allow some values
<code>debounce</code>	wait before sending values
<code>removeDuplicates</code>	skip repeating values
<code>flatMap</code>	transform + merge publishers
<code>combineLatest</code>	wait for multiple publishers

Example:

```

$textFieldValue
    .debounce(for: 0.5, scheduler: RunLoop.main)
    .removeDuplicates()
    .sink { value in
        print("Searching: \(value)")
    }

```

Perfect for:

- search bar behavior
- throttling API calls

8. Combine with URLSession (VERY IMPORTANT)

Instead of async/await:

```

struct User: Decodable {
    let name: String
}

func fetchUsers() -> AnyPublisher<[User], Error> {
    let url = URL(string: "https://
jsonplaceholder.typicode.com/users")!
    return URLSession.shared.dataTaskPublisher(for: url)
        .map(\.data)
        .decode(type: [User].self, decoder:
JSONDecoder())
        .eraseToAnyPublisher()
}

```

Explanation:

Step	What it does
<code>dataTaskPublisher</code>	publisher for URLSession
<code>.map(\.data)</code>	extract data
<code>.decode</code>	convert JSON to struct
<code>.eraseToAnyPublisher()</code>	hides internal type

Subscribing:

```
fetchUsers()  
    .sink(receiveCompletion: { print($0) },  
         receiveValue: { users in print(users) })  
    .store(in: &cancellables)
```

★ 9. Subjects (Manual Publishers)

PassthroughSubject

Publishes values manually.

```
let subject = PassthroughSubject<String, Never>()  
  
subject.send("Hello")  
subject.send("World")
```

CurrentValueSubject

Stores **latest value** + publishes updates.

```
let subject = CurrentValueSubject<Int, Never>(0)  
  
subject.send(1)  
subject.send(2)  
Used for:
```

- State management
- Live updates

★ 10. Handling Memory — Cancellables

Every subscription must be stored:

```
var cancellables = Set<AnyCancellable>()  
Then:
```

```
publisher
    .sink { print($0) }
    .store(in: &cancellables)
```

If you don't store it → subscription ends → no updates.

★ 11. Combine + SwiftUI Example

```
class SearchViewModel: ObservableObject {
    @Published var query = ""
    @Published var results: [String] = []

    private var cancellables = Set<AnyCancellable>()

    init() {
        $query
            .debounce(for: .milliseconds(300), scheduler:
RunLoop.main)
            .removeDuplicates()
            .sink { text in
                print("Searching:", text)
            }
            .store(in: &cancellables)
    }
}
UI:
```

```
struct SearchView: View {
    @StateObject var vm = SearchViewModel()

    var body: some View {
        TextField("Search", text: $vm.query)
            .padding()
    }
}
```

★ 12. Combine vs async/await (Interview Question)

Combine:

- Best for **continuous streams** (text fields, sockets, UI input)
- Works great with SwiftUI
- Has lots of operators

async/await:

- Best for **single async operations**
(1 API call → 1 result)

PERFECT INTERVIEW ANSWER:

“Use Combine when dealing with streams of data or multiple transformations.
Use async/await for simple, one-shot async calls.”

13. Combine Interview Questions (with strong answers)

Q1: What is a Publisher?

A type that emits values over time to its subscribers.

Q2: What is @Published?

A property wrapper that exposes a publisher (`$variable`) for UI updates.

Q3: What is AnyPublisher?

A type-erased publisher that hides underlying implementation.

Q4: How to avoid retain cycles in Combine?

Use `[weak self]` inside sink/assign.

Q5: What does `eraseToAnyPublisher()` do?

It hides the complex internal type, improving abstraction and flexibility.

? Q6: What's the difference between PassthroughSubject and CurrentValueSubject?

Passthrough sends only new values;
CurrentValue stores and sends the latest value.

★ 14. Combine Summary Table

Concept	Meaning
Publisher	Emits values
Subscriber	Receives values
@Published	Auto-updating state
sink	Subscribe to values
assign	Update a property
Operators	Transform the stream
Cancellables	Manage memory
Subjects	Manual publishers

2. CoreLocation

◆ What is it?

Provides:

- GPS location
- Geofencing
- Heading
- Region monitoring

◆ Permissions

`NSLocationWhenInUseUsageDescription`
`NSLocationAlwaysUsageDescription`

✅ Example

```
let manager = CLLocationManager()  
manager.requestWhenInUseAuthorization()  
manager.startUpdatingLocation()
```

◆ Delegate

```
func locationManager(_ manager: CLLocationManager,  
                    didUpdateLocations locations:  
                    [CLLocation]) {  
    print(locations.last!)  
}
```

🎯 Interview Answer

CoreLocation provides precise and background-aware location services.

3. CoreBluetooth

◆ What is it?

Bluetooth Low Energy (BLE) communication:

- Wearables
- IoT
- Medical devices

◆ Roles

Role	Meaning
Central	iPhone
Peripheral	Device

✅ Scan Devices

```
let manager = CBCentralManager(delegate: self, queue: nil)

func centralManagerDidUpdateState(_ central: CBCentralManager) {
    if central.state == .poweredOn {
        central.scanForPeripherals(withServices: nil)
    }
}
```

🎯 Interview Answer

CoreBluetooth handles BLE device discovery and data exchange.

4. AVFoundation

◆ What is it?

Handles:

- Audio
- Video
- Camera
- Media playback

◆ Use Cases

- Camera apps
- Voice recording
- Video streaming

✓ Play Audio

```
let player = AVAudioPlayer(contentsOf: url)
player.play()
```

◆ Camera Capture

```
let session = AVCaptureSession()
session.startRunning()
```

🎯 Interview Answer

AVFoundation provides low-level media capture and playback.

5. MapKit

◆ What is it?

- Maps
- Annotations
- Routes
- Geocoding

✅ Map Example

```
let mapView = MKMapView()  
mapView.showsUserLocation = true
```

◆ Add Annotation

```
let pin = MKPointAnnotation()  
pin.coordinate = CLLocationCoordinate2D(latitude: 28.6,  
longitude: 77.2)  
mapView.addAnnotation(pin)
```

🎯 Interview Answer

MapKit integrates Apple Maps with location and routing features.

6. PushKit / CallKit

◆ PushKit

- Used for **VoIP** pushes
- Wakes app for incoming calls

```
PKPushRegistry(queue: .main)
```

◆ CallKit

- Native call UI
- Integrates with Phone app

✅ Incoming Call

```
let update = CXCallUpdate()  
update.hasVideo = true  
provider.reportNewIncomingCall(with: uuid, update:  
update)
```

🎯 Interview Answer

PushKit wakes the app, CallKit displays system call UI.

7. BackgroundTasks

◆ What is it?

Runs **deferrable work** when app is backgrounded.

◆ Task Types

Task	Purpose
BGAppRefreshTask	Short refresh
BGProcessingTask	Heavy work

✓ Register Task

```
BGTaskScheduler.shared.register(  
    forTaskWithIdentifier: "com.app.refresh",  
    using: nil  
) { task in  
    task.setTaskCompleted(success: true)  
}
```

🎯 Interview Answer

BackgroundTasks allow system-controlled background execution.

8. StoreKit (In-App Purchases & Subscriptions)

◆ What is StoreKit?

StoreKit handles:

- Buying digital goods
- Subscriptions
- One-time purchases
- Restoring purchases
- App Store dialogues
- Receipt validation

★ 2. Types of In-App Purchases

Type	Example
Consumable	coins, tokens
Non-consumable	remove ads
Auto-renew subscription	monthly premium plan
Non-renewing	1-month pass

★ 3. High-Level Purchase Flow (Interview Roadmap)

App → StoreKit → App Store → User Pays → StoreKit → App
↓
Receipt is generated
↓
App or backend validates receipt

★ 4. Setting Up Products in App Store Connect

You configure:

- Product ID
- Type (consumable / non-consumable)
- Price
- Localized description

★ 5. Fetching Products (StoreKit 2 – Modern API)

Step 1: Request product from App Store

```
import StoreKit

let products = try await Product.products(for:
["com.example.premium"])
```

★ 6. Purchasing a Product

```
let product = products.first!

let result = try await product.purchase()

switch result {
case .success(let verification):
    print("Purchased")
case .userCancelled:
    print("Cancelled")
default:
    break
}
```

★ 7. Restoring Purchases

```
let result = try await AppStore.sync()
```

This retrieves previous purchases.

★ 8. Receipt Validation

✓ On-device validation (simple, insecure)

✓ Server-side validation (secure)

Server sends receipt to Apple's validation API:

<https://buy.itunes.apple.com/verifyReceipt>

Interview answer:

“Production servers use Apple's receipt validation endpoints to ensure purchases are legitimate.”

★ 9. Subscription Status (StoreKit 2)

```
for await status in product.currentEntitlement {  
    if status.isActive {  
        print("Subscription active")  
    }  
}
```

★ 10. Sandbox Testing (VERY important)

You test IAP using:

- Sandbox users
- TestFlight builds
- Lower prices for test environment

★ 11. Common IAP Interview Questions

? Q1: What is a consumable product?

Something that can be purchased multiple times (e.g., coins).

? Q2: What is the difference between restore purchase and repurchase?

Restore brings back **non-consumables** or **subscriptions** for free.
Consumables cannot be restored.

? Q3: Why validate receipts on server?

To prevent fraud and verify authenticity.

? Q4: Can you refund purchases?

No — handled only by Apple.

? Q5: What happens if network is lost during purchase?

StoreKit automatically retries or provides a failure callback.

★ 12. Combine In-App Purchases + Push Notifications (Advanced Use Case)

Example:

- User purchases premium
- Server receives receipt
- Server sends push notification → “Your premium is active”
- App updates UI

Companies love this scenario.

★ 13. Perfect One-Line Answers (Use in interview)

Push Notifications:

“Push notifications use APNs. The device registers for a token, the server sends notification payloads to APNs, and the OS delivers them to the user.”

In-App Purchases:

“StoreKit handles purchases and subscriptions. Receipt validation confirms the purchase. Consumables can be bought multiple times; non-consumables and subscriptions can be restored.”

9. Framework Usage Cheat Sheet

Framework	Use
Combine	Reactive streams
CoreLocation	GPS & geofencing
CoreBluetooth	BLE
AVFoundation	Media
MapKit	Maps
PushKit	VoIP push
CallKit	System calls
BackgroundTasks	Background work
StoreKit	Subscriptions

🔥 Power Interview Answer

I've worked with Combine for reactive flows, CoreLocation and MapKit for location features, AVFoundation for media, CoreBluetooth for BLE, PushKit/CallKit for VoIP, BackgroundTasks for background execution, and StoreKit for subscriptions.

Objective-C — Interview Crash Course

1 What is Objective-C? (Opening answer)

Objective-C is a dynamic, object-oriented language built on C, used historically for iOS/macOS development and still important for legacy codebases and system frameworks.

Most Apple frameworks were **originally** written in Objective-C.

2 Objective-C vs Swift (INTERVIEW FAVORITE)

Objective-C	Swift
Dynamic	Mostly static
Message passing	Method calls
Header + implementation	Single file
Verbose	Concise
Runtime-heavy	Compile-time safety

👉 Many companies still ask Objective-C to maintain legacy apps.

3 Basic Syntax (MUST KNOW)

Hello World

```
NSLog(@"Hello World");
```

Class Declaration

Header (.h)

```
@interface Person : NSObject
@property NSString *name;
- (void)sayHello;
@end
```

Implementation (.m)

```
@implementation Person
- (void)sayHello {
    NSLog(@"Hello %@", self.name);
}
@end
```

4 Message Passing (VERY IMPORTANT)

Objective-C does **not** call methods.
It **sends messages**.

```
[person sayHello];
```

Means:

“Send the message `sayHello` to `person`.”

5 Method with Parameters (syntax scares people)

```
- (void)greet:(NSString *)name age:(NSInteger)age;
```

Call:

```
[person greet:@"Manav" age:25];
```

➡ Method names include parameter names.

6 Properties & Memory Management

Property declaration

```
@property (nonatomic, strong) NSString *name;
```

Common property attributes

Attribute	Meaning
strong	Owns object
weak	No ownership

<code>assign</code>	For primitives
<code>copy</code>	For NSString, blocks
<code>nonatomic</code>	Faster, no thread safety

7 ARC (Automatic Reference Counting)

ARC automatically manages memory by tracking object ownership.

You **do NOT** write:

```
retain
release
dealloc
```

But you **must understand strong vs weak**.

Example

```
@property (nonatomic, weak) id delegate;
```

Used to **avoid retain cycles**.

8 nil Safety (Objective-C magic)

Calling a method on `nil`:

```
NSString *name = nil;
[name uppercaseString]; //  no crash
Returns nil.
```

👉 This is **VERY DIFFERENT** from Swift.

9 Protocols (similar to Swift)

```
@protocol Flyable <NSObject>
- (void)fly;
@end
```

Usage:

```
@interface Bird : NSObject <Flyable>
@end
```

10 Delegation Pattern (INTERVIEW GOLD)

Protocol

```
@protocol DownloadDelegate <NSObject>
- (void)didFinishDownload;
@end
```

Delegate property

```
@property (nonatomic, weak) id<DownloadDelegate>
delegate;
```

Used everywhere in UIKit.

1 1 Categories (VERY IMPORTANT)

Categories add methods to existing classes without subclassing.

```
@interface NSString (Utils)
- (BOOL)isEmpty;
@end
```

Used for:

- Utility methods
- Extensions
- Legacy code cleanup

1 2 Blocks (Objective-C Closures)

```
void (^completion)(NSString *) = ^(NSString *result) {
    NSLog(@"%@", result);
};
```

```
};
```

Used for:

- Callbacks
- Async code
- Networking

13 Objective-C Runtime (ADVANCED INTERVIEW)

Objective-C uses **runtime dispatch**.

```
objc_msgSend(object, selector)
```

This enables:

- Method swizzling
- Dynamic behavior
- KVO
- Core Data magic

14 Swift ↔ Objective-C Interoperability

Call Obj-C in Swift

```
#import "Person.h"
```

Call Swift in Obj-C

```
#import "ProjectName-Swift.h"
```

Swift needs:

```
@objc
```

15 When interviewers ask Objective-C

They expect you to know:

- Syntax basics
- ARC & memory management
- Delegates & protocols
- Categories
- Blocks
- Runtime awareness

✗ They don't expect you to write full apps.

One-line interview answers (memorize)

Q: Why Objective-C still relevant?

Legacy codebases and Apple frameworks.

Q: Biggest difference from Swift?

Dynamic runtime vs compile-time safety.

Q: What is nil messaging?

Safe no-op behavior when messaging nil.

Q: What is a category?

Adds methods to existing classes.

Mental model

Swift → safe, modern

Objective-C → dynamic, powerful, legacy

What is Core Animation? (plain English)

Core Animation is the engine that makes iOS UI smooth, fast, and animated.

Whenever you see:

- Smooth transitions
- Fade in / fade out
- Scaling, rotation
- Scrolling at 120 FPS

👉 **Core Animation is doing the work**, even if you never write its code directly.

Important truth (INTERVIEW GOLD)

UIKit animations and SwiftUI animations are built on top of Core Animation.

So when you write:

```
UIView.animate(withDuration: 0.3) {  
    view.alpha = 0  
}
```

You are actually using **Core Animation indirectly**.

The core concept: CALayer (VERY IMPORTANT)

UIKit view hierarchy

UIView

└─ CALayer ← Core Animation lives here

- Every UIView has a layer
- Animations happen on **layers**, not views

```
view.layer.cornerRadius = 10  
view.layer.shadowOpacity = 0.5
```

👆 This is **Core Animation**

Why Core Animation is fast

Animations run on the GPU, not the CPU.

- CPU → logic, calculations
- GPU → drawing pixels

Core Animation:

- Sends animation instructions to GPU
- GPU runs animations independently
- UI stays responsive

That's why animations don't block the main thread.

Types of Core Animation (hands-on)

1 Implicit Animations (automatic)

```
view.layer.opacity = 0
```

If this runs **inside an animation context**, Core Animation animates it automatically.

UIKit wraps this for you:

```
UIView.animate(withDuration: 0.3) {  
    view.layer.opacity = 0  
}
```

✓ You didn't create an animation

✓ Core Animation did

2 Explicit Animations (direct Core Animation)

Used when you want **full control**.

```
let animation = CABasicAnimation(keyPath: "opacity")  
animation.fromValue = 1  
animation.toValue = 0
```

```
animation.duration = 0.3
```

```
view.layer.add(animation, forKey: "fade")
```

Used for:

- Complex animations
- Performance-critical UI
- Custom drawing

3 Property animations (most common)

```
UIView.animate(withDuration: 0.3) {  
    view.transform = CGAffineTransform(scaleX: 1.2, y:  
1.2)  
}
```

Under the hood:

- UIKit → Core Animation → GPU

Core Animation properties you should know (INTERVIEW)

```
layer.opacity  
layer.position  
layer.bounds  
layer.transform  
layer.cornerRadius  
layer.shadowOpacity  
layer.shadowRadius
```

If you know these → interviewer knows you've worked with UI deeply.

Why Core Animation matters in real apps

Used in:

- Smooth scrolling lists

- Button press effects
- Skeleton loaders
- Shimmer effects
- Charts & graphs
- Custom loaders

Apps without smooth animation feel **cheap**.

Common real-world example (hands-on)

Button tap animation

```
UIView.animate(withDuration: 0.1,
               animations: {
    button.transform = CGAffineTransform(scaleX: 0.95, y:
0.95)
}) { _ in
    UIView.animate(withDuration: 0.1) {
        button.transform = .identity
    }
}
```

This is **Core Animation-powered UI feedback**.

Core Animation vs Core Data (interview contrast)

Core Animation	Core Data
UI rendering	Data storage
GPU-driven	Disk + memory
Visual	Persistence
Smooth UX	Efficient data

Both are **low-level Apple frameworks**.

One-line interview answer (USE THIS)

Core Animation is the low-level framework responsible for rendering and animating UI efficiently on iOS using the GPU.

How interviewers check “hands-on” knowledge

They expect you to know:

- CALayer exists
- Animations don't block UI
- UIKit animations use Core Animation
- GPU vs CPU difference

Not math-heavy animation formulas.

Mental model (remember forever)

You change UI →
UIKit →
Core Animation →
GPU →
Smooth animation

UIKit — Interview & Practical Guide

1 What is UIKit? (opening answer)

UIKit is Apple's UI framework used to build iOS apps using views, view controllers, and an imperative programming model.

It is the **original and still widely used** UI framework on iOS.

2 UIKit vs SwiftUI (VERY COMMON QUESTION)

UIKit	SwiftUI
Imperative	Declarative
ViewControllers	Views
Manual updates	State-driven
Mature & stable	Modern & evolving
Heavy boilerplate	Minimal code

👉 Most production apps still use UIKit or a mix of both.

3 Core building blocks of UIKit

◆ UIView

A rectangular area on screen.

Examples:

UILabel

UIButton

UIImageView

UITableView

All UI elements inherit from UIView.

◆ UIViewController (VERY IMPORTANT)

Manages a screen and its lifecycle.

```
class HomeViewController: UIViewController {  
    override func viewDidLoad() {  
        super.viewDidLoad()  
    }  
}
```

Every screen = **ViewController**

4 View Controller Lifecycle (INTERVIEW GOLD)

```
init  
↓  
loadView  
↓  
viewDidLoad  
↓  
viewWillAppear  
↓  
viewDidAppear  
↓  
viewWillDisappear  
↓  
viewDidDisappear
```

Common usage:

- `viewDidLoad` → setup UI, API calls
- `viewWillAppear` → refresh data
- `deinit` → cleanup

5 Layout in UIKit

◆ Auto Layout (MOST IMPORTANT)

UIKit uses **constraints** to define layout.

```
label.translatesAutoresizingMaskIntoConstraints = false
```

Constraints define:

- Position
- Size
- Relation to other views

◆ Stack Views (easy layout)

```
UIStackView(arrangedSubviews: [label, button])
```

- Vertical / Horizontal
- Auto Layout friendly
- Cleaner code

6 Navigation in UIKit

◆ UINavigationController

```
navigationController?.pushViewController(vc, animated: true)
```

- Stack-based navigation
- Back button handled automatically

◆ Tab Bar Controller

```
UITabBarController
```

- Bottom tabs
- Multiple root screens

7 TableView & CollectionView (VERY IMPORTANT)

UITableView

Used for:

- Lists
- Settings
- Chat screens

Key methods:

```
numberOfRowsInSection  
cellForRowAt  
didSelectRowAt
```

UICollectionView

Used for:

- Grids
- Carousels
- Complex layouts

More powerful than table view.

8 Target–Action pattern

UIKit uses **Target–Action** for events.

```
button.addTarget(self,  
                 action: #selector(buttonTapped),  
                 for: .touchUpInside)
```

This is how UIKit handles user interaction.

9 Delegation pattern (UIKit backbone)

UIKit heavily uses **delegates**.

Examples:

- UITableViewDelegate

- UITextFieldDelegate
- UIScrollViewDelegate

Pattern:

“I don’t know how to handle this — you decide.”

10 Memory Management in UIKit

UIKit uses **ARC**.

Common rules:

- Use **weak** for delegates
- Avoid retain cycles in closures

`weak var delegate: SomeDelegate?`

11 Threading & UIKit (INTERVIEW TRAP)

! All UI updates must happen on the main thread

```
DispatchQueue.main.async {
    self.label.text = "Updated"
}
```

Failure → crashes / UI bugs.

12 Animations in UIKit

UIKit animations are built on **Core Animation**.

```
UIView.animate(withDuration: 0.3) {
    view.alpha = 0
}
```

Smooth, GPU-powered animations.

1 3 Storyboards vs Programmatic UI

Storyboards

- Visual
- Faster for small apps
- Merge conflicts

Programmatic UI

- Scalable
- Better for teams
- Preferred in interviews

1 4 UIKit + MVC architecture

UIKit apps usually follow MVC:

Model → Data

View → UI

Controller → Logic

But leads to **Massive View Controllers**.

Modern apps use:

- MVVM
- Coordinator pattern

1 5 UIKit + SwiftUI (modern reality)

UIKit and SwiftUI **coexist**.

```
UIHostingController(rootView: SwiftUIView())
```

Used in:

- Incremental migration

- Legacy apps

Interview one-liner (USE THIS)

UIKit is an imperative UI framework for iOS that uses view controllers to manage views, handle user interaction, and control app navigation.

Mental model (remember forever)

UIKit =
Views + ViewControllers + Delegates + AutoLayout

What interviewers expect from “UIKit knowledge”

They expect you to know:

- ViewController lifecycle
- Auto Layout basics
- TableView / CollectionView
- Delegates
- Navigation
- Main-thread UI rule

Not pixel-perfect UI code.

Final confidence boost 💪

If you’ve:

- Built screens with UINavigationController
- Used TableView or CollectionView
- Handled navigation
- Fixed layout issues

👉 You **know** UIKit.

